

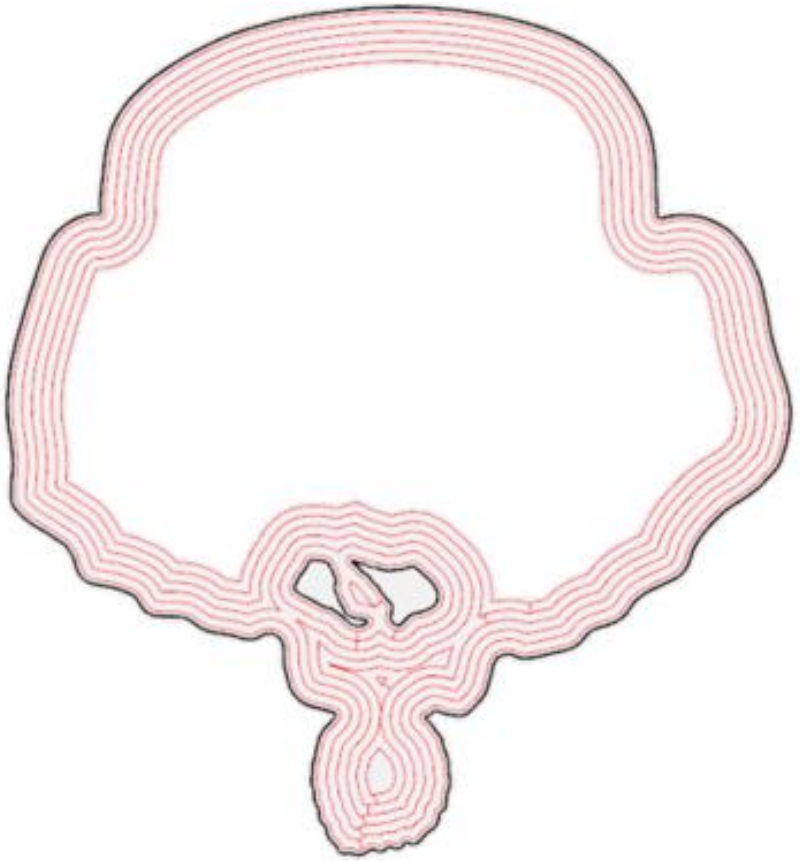
计算机辅助制造实践

——Python实现三维打印路径规划

浙江大学机械工程学院 林志伟
玉泉教1-215　zjtzhylin1986@163.com

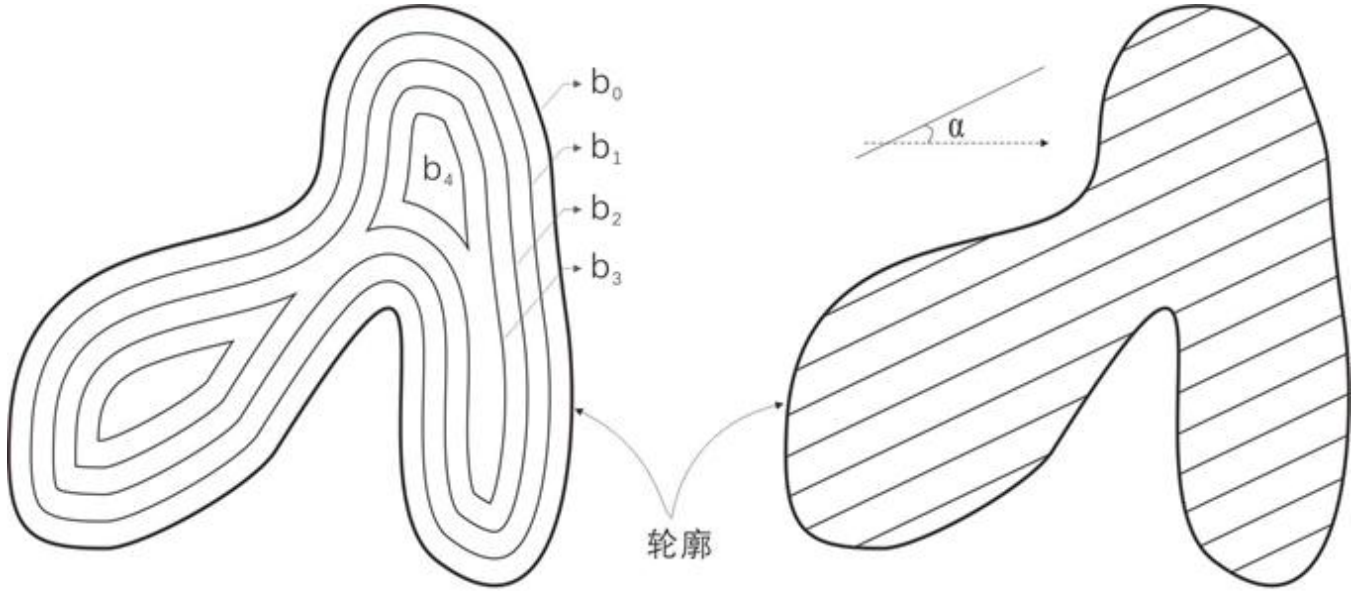
第8章 轮廓平行填充路径生成

- 常用填充路径概述
- 轮廓偏置基础方法简介
- 基于裁剪的轮廓偏置
- Clipper适配器构建
- 轮廓路径生成与连接
- 轮廓路径生成测试



1. 常用填充路径概述

- **三维打印工艺特点**：“**降维**”制造，沿一个方向**逐层堆叠生长。**
- **路径规划作用**：将复杂的三维模型转化为一系列相对简单的二维填充路径（toopath）。
- **填充路径要求**：1）满足工艺需求；2）覆盖加工区域。
  - 理论上，对任意形式的曲线，只要**满足工艺要求**，且能**覆盖**整个**打印区域**，那么它就是一种有效的三维打印填充路径形式。
- **常用填充路径**：1）轮廓平行（contour-parallel）填充路径；2）方向平行（direction-parallel）填充路径。



(a) 轮廓平行填充路径　　(b) 方向平行填充路径

1. 常用填充路径概述

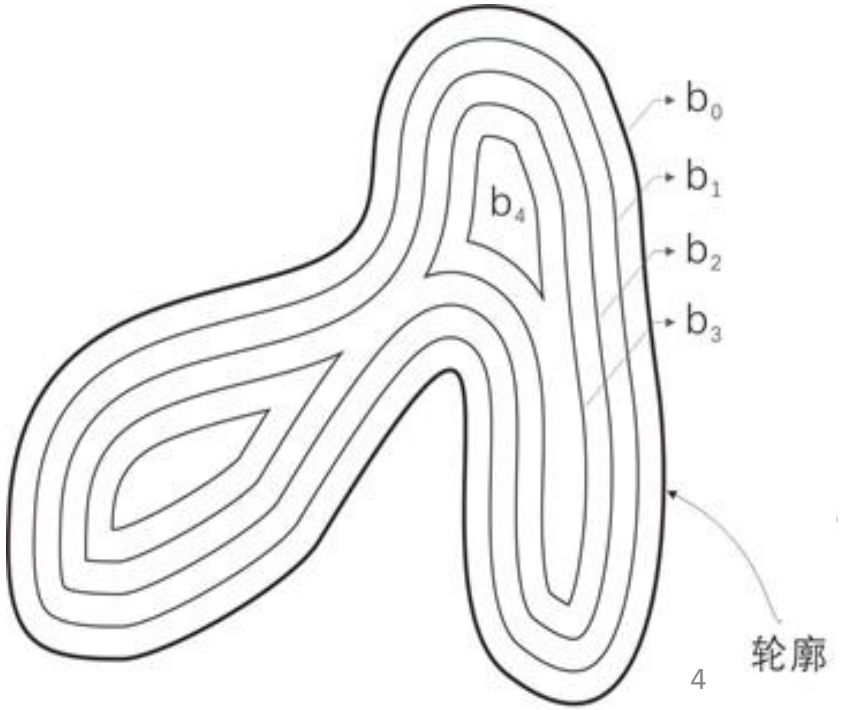
• 轮廓平行填充路径

- 轮廓平行填充路径（简称**轮廓填充路径**或**轮廓路径**）实质上就是一系列平面偏置曲线。这些偏置曲线通过对表示打印区域的边界轮廓连续偏置生成。
- 如果两条曲线存在**偏置（等距偏置）**关系，那么就认为这两条曲线是“平行”的，这是称偏置路径为轮廓平行路径的原因。
- **等距偏置是三维打印的一个工艺需求**，如在FDM三维打印中，从喷头挤出的丝宽是不变的，理论上，要把这些不变宽度的丝材**无缝地平铺**在打印区域内，就必须用到等距篇置。**（什么是“等距偏置”？什么是“非等距偏置”？）**

轮廓平行路径生成案例分析

- b_0表示打印区域边界轮廓，即切片轮廓，b_0包含若干外边界，也可能包含若干内边界。按照约定，外边界轮廓方向为逆时针，内边界轮廓方向为顺时针，即外逆内顺。
- b_1同样表示一个曲线序列，它保存了对b_0中的各轮廓进行偏置得到曲线。
- 同理，b_2为对b_1偏置得到的曲线序列，b_3为对b_2偏置得到的曲线序列……
- 在连续偏置过程中，b_k（k = 1, 2, 3…）可能会分裂成多条曲线（如图中b_3），因此，b_k中曲线的数量会越来越多，但由它包围的范围会逐渐缩小，并最终收敛于一点（或多点），从而填充整个打印区域。

思维拓展：如果求任意轮廓的最大内接圆?



1. 常用填充路径概述

- **组合路径（实际打印路径）**

□ 打印区域边缘，填充轮廓路径

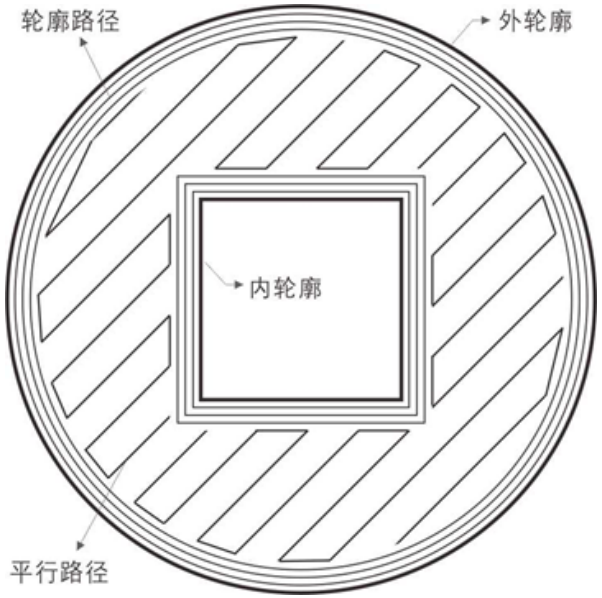
- 轮廓路径能够非常好地**跟随、还原**轮廓形状，从而保证被打印模型的外观精度。
- 但由于轮廓路径形状复杂，弯曲频繁，路径上曲率变化通常较大，最终导致打印机各轴电机**频繁加减速**，因此相对来说，轮廓路径的打印效率不高。

——保证精度

□ 打印区域内部，填充平行路径

- 平行路径在边缘处精度不高，在还原轮廓原貌时会出现难以避免的**锯齿**现象。
- 在每条平行线段内部，**路径的方向是恒定不变**的，打印机能始终以设定的**最高打印速度**进给，从而保证打印效率。

——保证效率

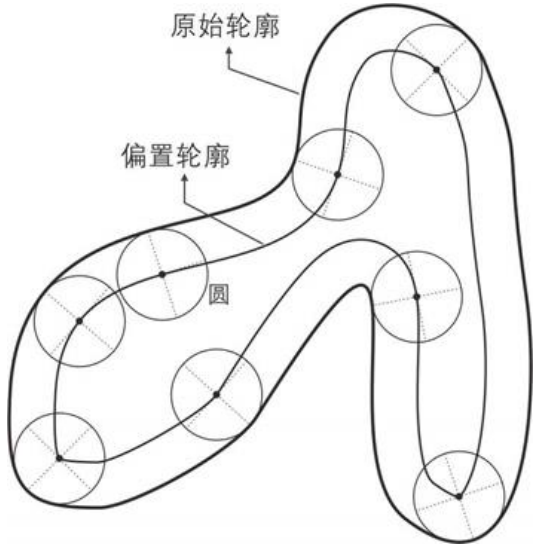


取长补短
综合优化利用

2. 轮廓偏置基础方法简介

- **生成轮廓填充路径的基础**是平面曲线的**等距偏置**（以下简称偏置）算法。
- 针对STL模型，因为得到的切片轮廓都是平面简单多边形，即不存在自相交的平面多边形，因此生成轮廓路径的基础是**平面简单多边形**的**偏置算法**。
- **平面简单多边形偏置**

  - **数学描述**：对一个输入多边形，在其一侧生成一个（或多个）新多边形，使得新多边形上每点到原始多边形的距离为给定的偏置距离。
  - **操作描述**：对一半径等于给定偏置距离的圆，使其沿着多边形一侧滚动，得到的圆心轨迹就是所求的偏置多边形。



看起来简单
实则很难的问题

2. 轮廓偏置基础方法简介

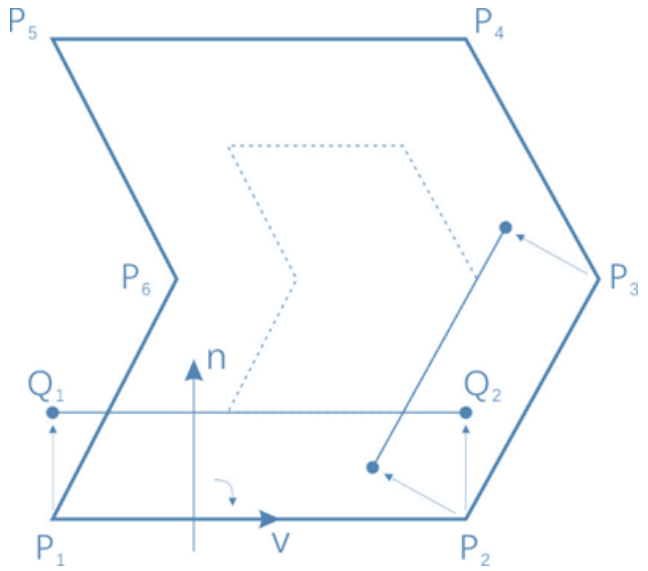
• 2.1. 基于线段平移的偏置

- **基本思路**：
  1. 把原始多边形拆分开来，当圆沿着它的其中一条边（线段）滚动时，此时圆心的轨迹就是一条线段，即偏置线段。
  2. 对原始多边形的每条边进行偏置。
  3. 有序地连接这些偏置线段就可得到原始多边形的偏置多边形。
- **线段偏置**：对线段的两个端点分别沿着线段法向平移给定的偏置距离。
  - 在GeomBase模块的Vector3D类中已经封装了**getOrthoVector2D**函数，作用就是求取当前向量在XY平面内的正交向量（单位向量），即法向量。
  - 假设输入线段的两端点分别为P_1、P_2，那么它的法向量可由以下代码求得：

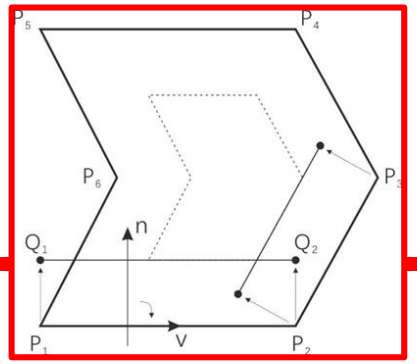
	线段P_1P_2的偏置线段计算方法		**8-1**
1	v = P1.pointTo(P2)	# v为直线P1P2方向	
2	n = v. getOrthoVector2D()	# n为直线P1P2法向量，n为单位向量	

法向量方向不确定性

- 线段P_1P_2事实上有两个法向量，分别指向直线的两侧。
- 如果要求取原始多边形的向内偏置多边形，且原始多边形方向为逆时针的话，此时要求的线段P_1P_2的法向应该指向P_1P_2的左侧。
- 线段左边、右边怎么定义？
- **getOrthoVector2D**函数返回的方向是不确定的。



2. 轮廓偏置基础方法简介



• 2.1. 基于线段平移的偏置：线段偏置

□ XY平面上如何确保向量n在线段P_1P_2的左侧?

- 对n和P_1P_2的方向向量（即v）进行**叉乘**（注意，n在前，v在后，向量叉乘特别需要注意两个向量前后顺序）；对得到的向量，根据向量叉乘的右手螺旋法则：
  - 如果它的方向指向坐标系Z轴负向，那么可以判定向量n指向线段P_1P_2左侧，此时无需对n进行任何处理；
  - 反之，如果它的方向指向Z轴正向，那么可以判定向量n指向线段P_1P_2右侧，此时需要对n进行取反处理。

```
3    if n.crossProduct(v).dz > 0:          # 如果 n 和 v 的叉乘向量指向正Z向
4      n.reverse()                         # 则对 n 方向取反
```

□ 偏置线段计算

- 记给定的偏置距离为d，线段P_1P_2两端点沿单位向量n平移后的点分别为Q_1、Q_2，则Q_1、Q_2可由以下代码计算得到：

```
5    Q1 = P1 + n.amplified(d)              # Q1 为 P1 的偏置点
6    Q2 = P2 + n.amplified(d)              # Q2 为 P2 的偏置点
```

  - 如果多边形为顺时针方向，而以上代码不变，那得到的偏置线段将在多边形的外侧。
  - 类似地，如果需要计算一个逆时针多边形每条边的向外偏置线段，需要调整以上代码中向量n的方向，使其总是在线段走向的右侧。

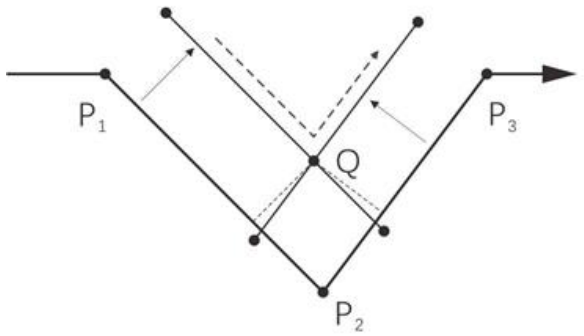
2. 轮廓偏置基础方法简介

- **2.1. 基于线段平移的偏置：偏置线段修补**
- 直接将偏置线段有序放在一起是无法得到封闭的偏置多边形的，需要**修补**！

如何判断
优角、劣角

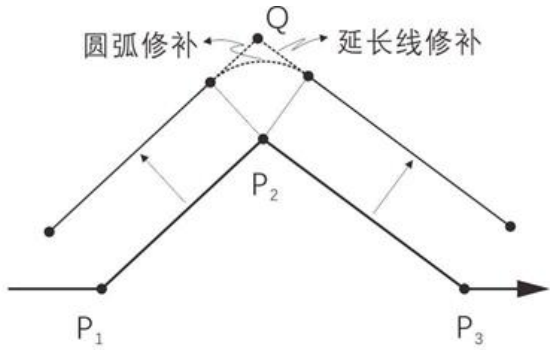
优角（小于180度）

对应的两条偏置线段必然相交，此时需要计算两偏置线段的交点，并在交点处对它们进行裁剪、连接。



(a) $\angle P_2 < 180°$

邻边所夹内角



(b) $\angle P_2 > 180°$

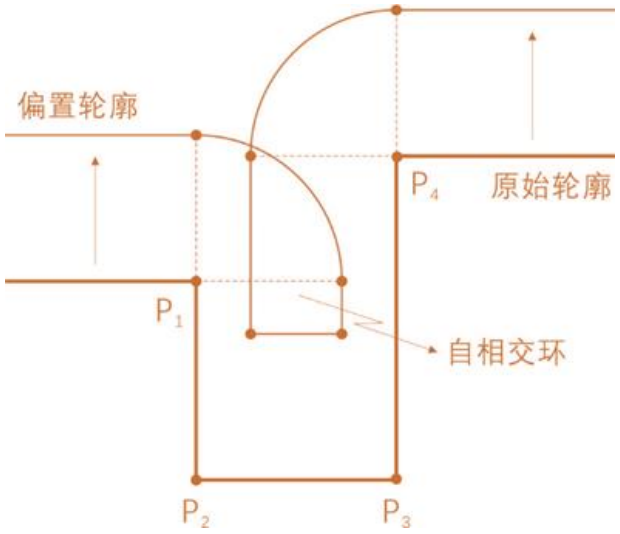
劣角（大于180度）

对应的两条偏置线段必然存在缝隙，需要弥补这个缝隙，比如分别延长两条偏置线段然后在其交点处相连，或使用一段半径为偏置距离d的圆弧衔接这两条偏置线段。

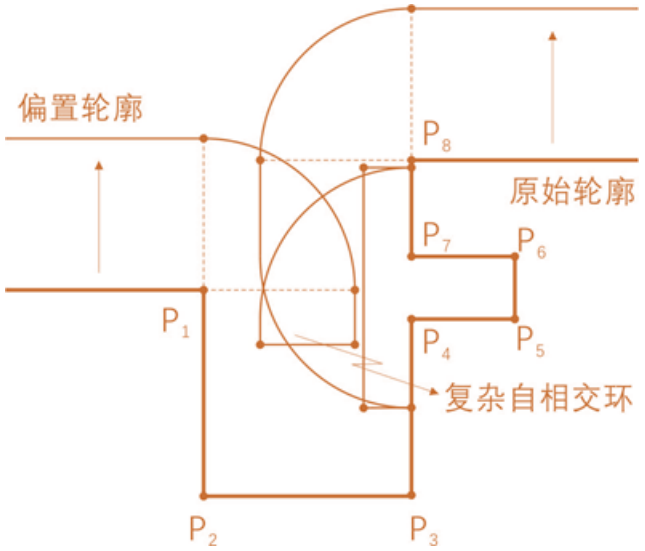
当用延长线去衔接偏置线段时，延长线上的点到原始轮廓上的距离将大于给定偏置距离，这不符合偏置线的定义，但是允许这种情况的存在。

2. 轮廓偏置基础方法简介

- **2.1. 基于线段平移的偏置：局部自相交环**
- 在偏置距离较小时，可运用以上修补方法生成它的偏置轮廓；但当偏置距离较大时，如果依然运用以上修补方法，将得到一个带有**自相交环**的偏置轮廓。
- 自相交环上的点到原始轮廓的距离显然是小于给定的偏置距离的，自相交环是偏置错误的表现，必须避免或消除。
- 如何消除偏置轮廓上的自相交环？
  - **一种思路**：**先计算偏置轮廓的交点，然后在交点处裁剪轮廓。是否可行？**



偏置轮廓上的局部自相交环



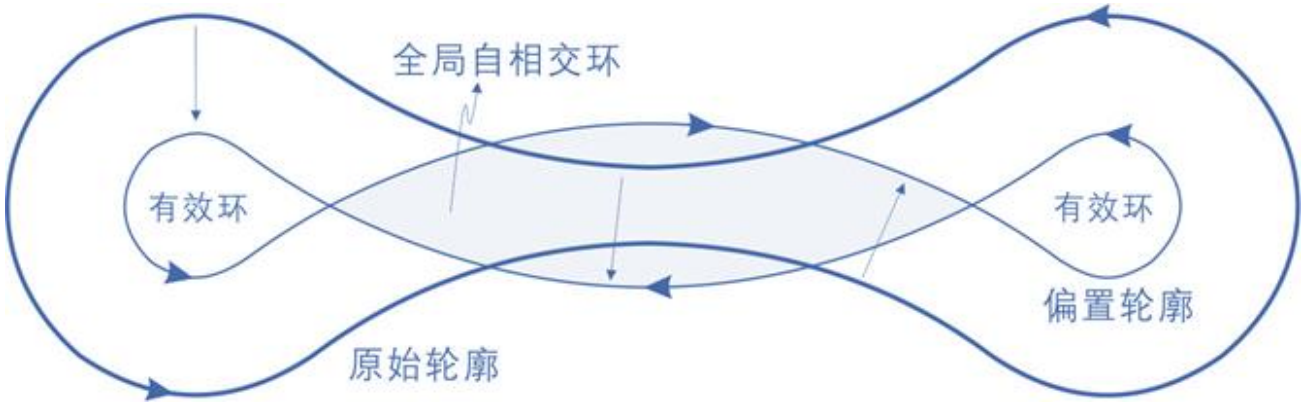
偏置轮廓上的复杂自相交环

2. 轮廓偏置基础方法简介

- **2.1. 基于线段平移的偏置：全局自相交环**

- 全局自相交环似乎是偏置过程中的合理存在，它上面的点到对应的原始轮廓距离是满足偏置要求的，但它其实也是一种偏置错误，因为它到另一部分原始轮廓的距离小于偏置距离。

- **全局自相交环的消除的常规思路**：
  - 首先两两遍历计算偏置轮廓边的所有交点；
  - 然后根据交点对偏置轮廓分段，并根据每段偏置轮廓到原始轮廓的距离是否小于偏置距离决定这段轮廓是否需要保留，
  - 最后按照**一定的逻辑（如轮廓段走向）**拼接保留的偏置轮廓段得到若干封闭偏置多边形。



偏置轮廓上的全局自相交环

2. 轮廓偏置基础方法简介

- **2.2. 基于角平分线的偏置（顶点平移）**
- **基本思路**

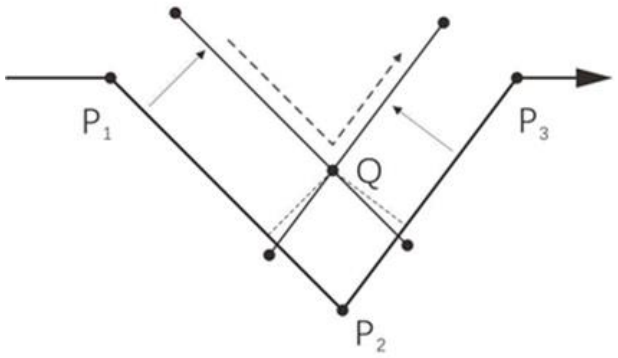
点Q的位置

两偏置线段（或延长线）之间的交点Q到原始线段P_1P_2、P_2P_3的距离均等于偏置距离，据此，不难判断**交点Q位于角$P_1P_2P_3$的角平分线上**。

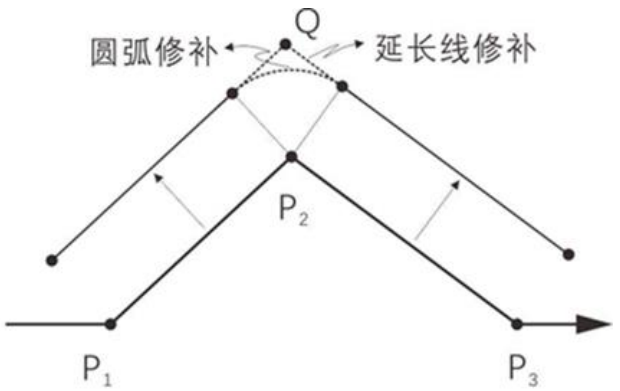
定义：

点Q为顶点P_2对应的偏置点。

只要依次连接多边形上的所有偏置点，就能直接得到一条初始偏置轮廓。



(a) $\angle P_2 < 180°$



(b) $\angle P_2 > 180°$

2. 轮廓偏置基础方法简介

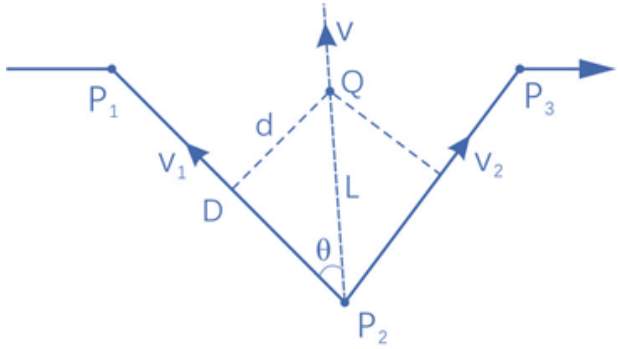
- **2.2. 基于角平分线的偏置：偏置点计算** ——以逆时针多边形的向内偏置为例

- **角平分线方向计算**：对原始轮廓上的3个连续点P_1、P_2、P_3，分别计算P_2P_1、P_2P_3的单位向量（记为v_1、v_2），然后将其相加就可得到角平分线的方向向量（也是单位向量，记为v），如下代码：

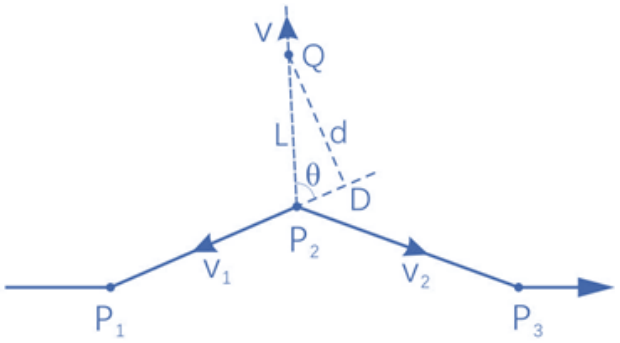
	偏置点计算方法	8-2
1	v1 = P2.pointTo(P1).normalized()	# v1为P2P1方向
2	v2 = P2.pointTo(P3).normalized()	# v2为P2P3方向
3	v = (v1 + v2).normalized()	# v 为角P1P2P3角平分的方向向量

- **角平分线方向修正**：当P_1、P_2、P_3所夹内角为优角时，v的方向是对的；但当3点所夹内角为劣角时，v的方向就是错的，需要对v进行取反操作。对向量v_1、v_2进行叉乘，如果得到的向量指向Z轴负向，则P_1、P_2、P_3所夹内角为优角，反之则3点所夹内角为劣角。

```
4   If v1.crossProduct(v2).dz > 0      # 如果 v1、v2叉乘向量指向Z轴正向
5     v.reverse()                      # 则对它们的和向量方向取反
```



(a) $\angle P_2 < 180°$



(b) $\angle P_2 > 180°$

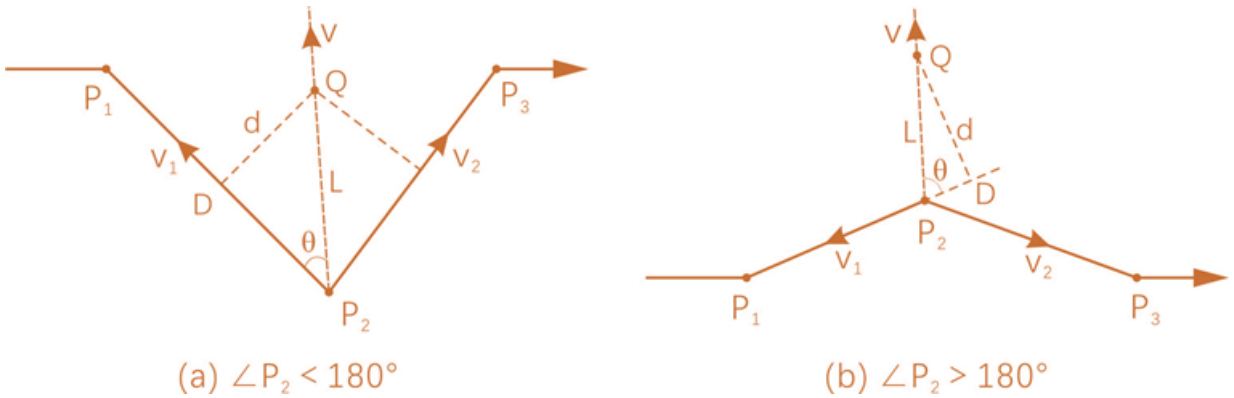
2. 轮廓偏置基础方法简介

- **2.2. 基于角平分线的偏置：偏置点计算** ——以逆时针多边形的向内偏置为例

- **角平分线上距离计算**：记P_2对应的偏置点为Q。Q在P_1P_2上的投影为D，在三角形QDP_2中，QD长度为已知量（即给定的偏置距离），只需知道角QP_2D的角度（记为θ，即theta），便可算出斜边P_2Q的长度（记为L），从而得到偏置点Q的坐标。

- 需要说明的是，尽管以上推导是针对优角情况做出，但同样适用于劣角情况。综上，计算偏置点的具体代码如下：

```
6    theta = v1.getAngle(v2) / 2          # 三角形CDP2内角CP2D角度
7    L = d / math.sin(theta)              # 三角形CDP2内斜边P2C长度
8    Q = P2 + v.amplified(L)              # 偏置点C坐标
```



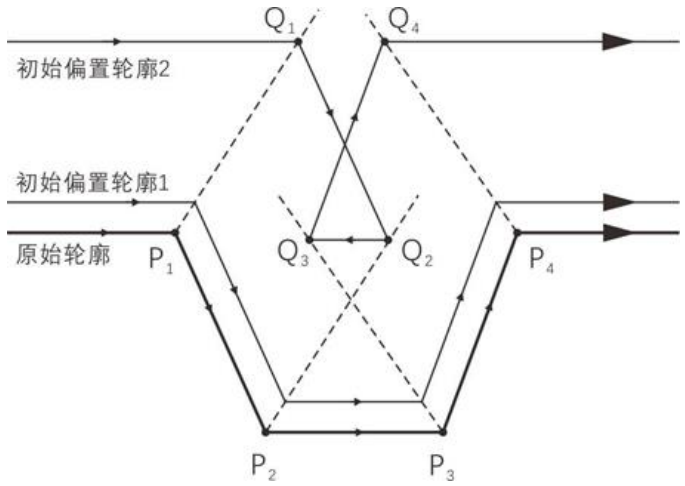
- 按以上步骤，计算一个给定多边形任意顶点的偏置点，然后把这些顶点有序连接起来，就可以得到原始多边形的**初始偏置多边形**。

2. 轮廓偏置基础方法简介

• 2.2. 基于角平分线的偏置：局部自相交环

根据偏置点位置识别、定位局部自相交环

- 如图，当偏置距离较大时，将会出现在偏置轮廓上出现一个局部自相交环。
- 对偏置点Q_1~Q_4，Q_1和Q_4到原始轮廓的距离大于给定的偏置距离，因此它们是合法的偏置点；但Q_2和Q_3到原始轮廓的距离小于给定的偏置距离，因此它们是非法的偏置点。
- 在有非法偏置点的附近，必然会有非法的偏置轮廓，因此原则上在定位非法偏置点后，就可找到那些非法的自相交环。
- 但是这种基于偏置点位置去识别自相交环的方法计算量较大，因为这种方法需要遍历计算每个偏置点到原始轮廓的距离，其算法时间复杂度至少是$O(n^2)$，其中n为原始多边形上顶点个数。

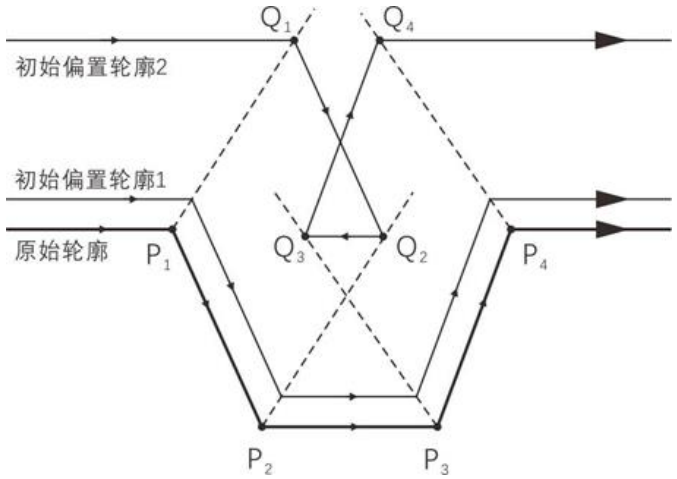


2. 轮廓偏置基础方法简介

- **2.2. 基于角平分线的偏置：局部自相交环**

根据偏置线方向识别、定位局部自相交环

- 在偏置过程中，偏置线段的方向也应该和与之对应的原始轮廓上的边的方向保持一致。
- 如图，偏置线段Q_1Q_2、Q_3Q_4分别和原始线段P_1P_2、P_3P_4的方向是一致的，因此Q_1Q_2和Q_3Q_4是合法的偏置线段；但是Q_2Q_3和P_2P_3的方向刚好相反，因此断定偏置线段Q_2Q_3必然是有问题的，即非法的。
- 在非法偏置线段附近，必然存在非法的偏置轮廓，据此，原则上可以找到自相交环。
- 相对偏置点位置识别法，偏置线方向识别法效率更高，它的算法时间复杂度为O(n)，其中n为原始多边形上顶点个数。

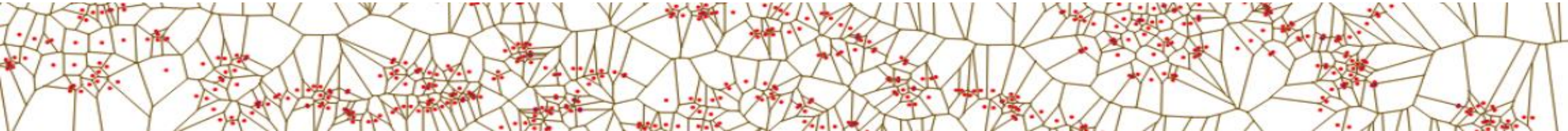


如何消除局部自相交环?

3. 基于裁剪的轮廓偏置

常用平面多边形偏置库

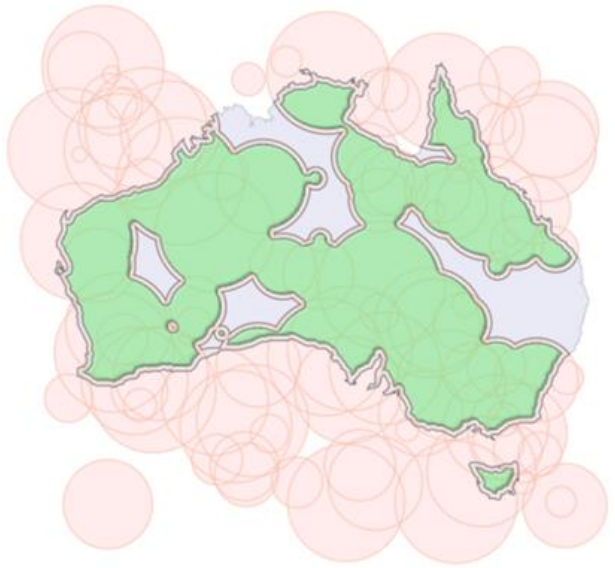
- **CGAL**
  - CGAL（https://www.cgal.org/）是一个重量级的大型计算几何库，包含了计算几何领域的诸多算法，比如凸壳生成、实体布尔运算、Delaunay三角化、Voronoi图、几何形状重建等。
  - 但CGAL源码采用C++模板技术编写而成，晦涩难懂，对初学者来说，阅读和使用难度较大。
- **Clipper**
  - Clipper（http://www.angusj.com/delphi/clipper.php）是一个轻量级的小型几何库，专门用于平面多边形的布尔运算，算法运行效率高、鲁棒性强。
  - 目前，Clipper已在多种编程语言上实现了直接可用的源代码，如C++、C#、Delphi、Python等。
- **……**

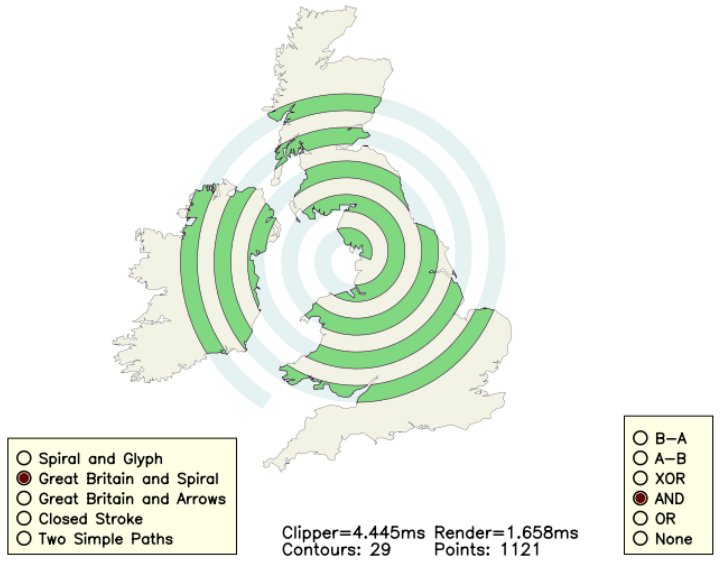


3. 基于裁剪的轮廓偏置

- **3.1. Clipper库简介**

- **Clipper**是一个专门用于平面闭合或非闭合多段线布尔（或叫裁剪）运算的开源计算几何库，可以实现平面图形的：**交**（intersection或and）、**并**（union或or）、**差**（difference）、**异或**（xor）以及**偏置**等运算功能。
- Clipper库由**Angus Johnson**根据Bala R. Vatti在1992年发表的“***A generic solution to polygon clipping***”论文完善实现，因此，Clipper库中的裁剪算法又叫Vatti’s clipping algorithm。
- 目前，最新版本的Clipper发布于2017年2月份，版本号为6.4.2。





3. 基于裁剪的轮廓偏置

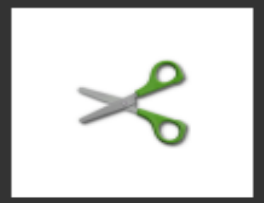
• 3.1. Clipper库简介：安装

方式一、下载Python版源文件

- Clipper库的源文件可以通过Clipper官网上的链接下载，它被托管在SourceForge网站上。在下载的压缩包中，可直接拷贝里面的clipper.py 文件到的目标项目目录，并在相应代码文件中通过 “from clipper import *”语句导入Clipper库。
- 相对C++版本的源代码，Python版本的Clipper库源代码更新较慢，目前能够下载到的最新Python版Clipper库的版本号是5.1.6

方式二、命令行： pip install pyclipper

- Python将自动下载版本合适且已经编译好的Clipper库文件，相比第一种方法，这种方法更加方便。
- 使用时，在相应代码文件中通过 “import pyclipper”语句导入Clipper库。
- PyPI（Python第三方库仓库）已经将Clipper库纳入了日常维护管理（参见https://pypi.org/project/pyclipper/），这从另一个侧面反映了Clipper库的通用性。
- 通过这种方式安装的Clipper库的版本号基本能和最新的版本号保持一致，因此**建议采用这种方式安装并使用Clipper库**。



3. 基于裁剪的轮廓偏置

- **3.1. Clipper简介：基本概念——Path**
- 在Clipper库中，所有算法的输入和输出都是**Path**的对象。
- **Path**和**Polyline**类似，不过Polyline三维多段线，而Path是二维多段线。
- Path可以包含一个点、两个点或多个点，它可以闭合，也可以不闭合，甚至还允许自相交，方便起见，在描述时仍然称Path为轮廓。
- Path中的点的类型为**IntPoint**，即其x、y坐标均为**整数类型（int）**！

1. Path为什么使用 IntPoint 作为点的基础结构?

2. 如何实现 Path 和 Polyline 的互相转化?

3. 基于裁剪的轮廓偏置

- **3.1. Clipper简介：基本概念——填充规则**

- 对由一条或多条闭合轮廓组合形成的平面封闭区域，Clipper定义了四种**填充规则**：
  - ✓ **奇偶填充** **Even-Odd**
  - ✓ **非零填充** **None-Zero**
  - ✓ **正填充** **Positive**
  - ✓ **负填充** **Negative**

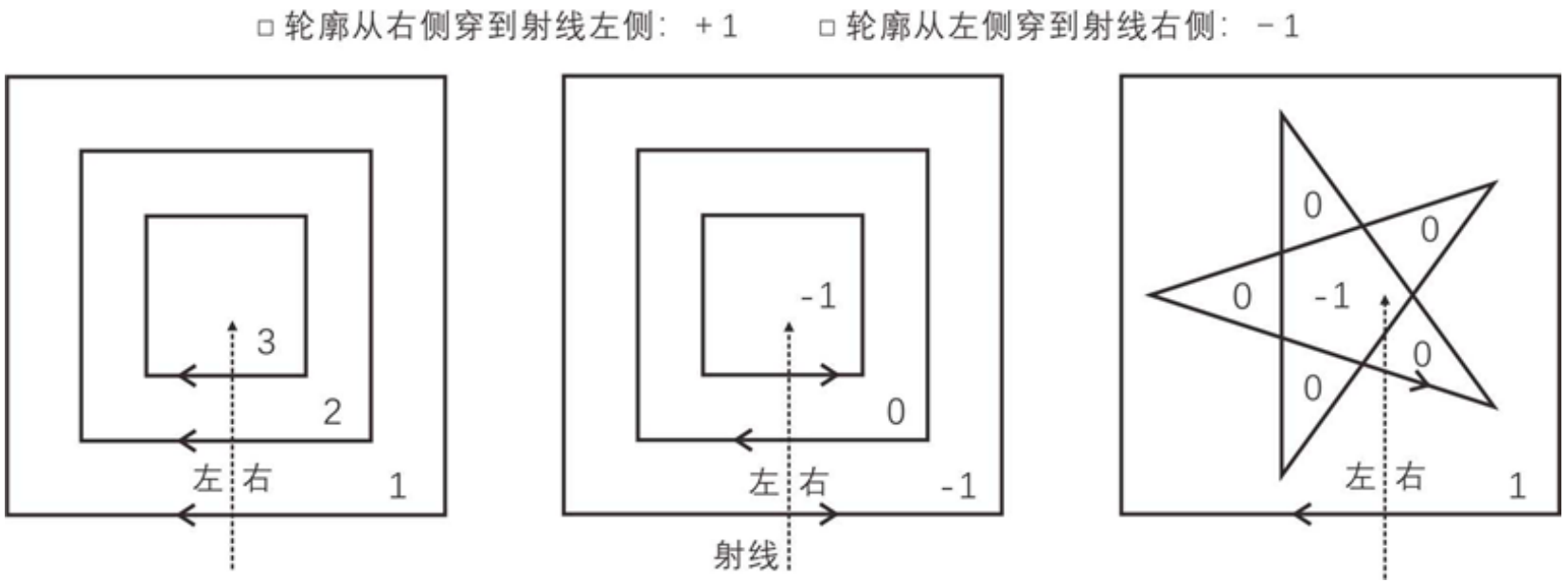
说明

这里的填充是计算机图形学的概念，这些规则定义了平面封闭区域内哪些区域需要被填充，哪些区域不需要被填充。

- **奇偶填充** **——最简单的填充方式**
  - 对一条起点在区域外射线，假想它逐渐伸长和区域各条边界轮廓相交，并最终穿越区域的场景。在这一过程中，和射线相交的第奇数个轮廓内部区域是填充区域，第偶数个轮廓内部区域是非填充区域。
  - 奇偶填充一般适用于由那些轮廓本身不存在自相交情况且轮廓之间也不存在相交情况的轮廓围成的平面区域。

3. 基于裁剪的轮廓偏置

- **3.1. Clipper简介：基本概念——填充规则**
- **非零填充**、**正填充**、**负填充**的概念相对复杂，它们根据射线穿过区域轮廓时的**绕数（winding number）**来确定某一区域是否需要填充。
- **绕数概念**：如图，如果区域中的某条轮廓**从右到左**穿过射线，那么这条轮廓的内部区域绕数**加1**；如果轮廓**从左到右**穿过射线，那么相应区域绕数**减1**。

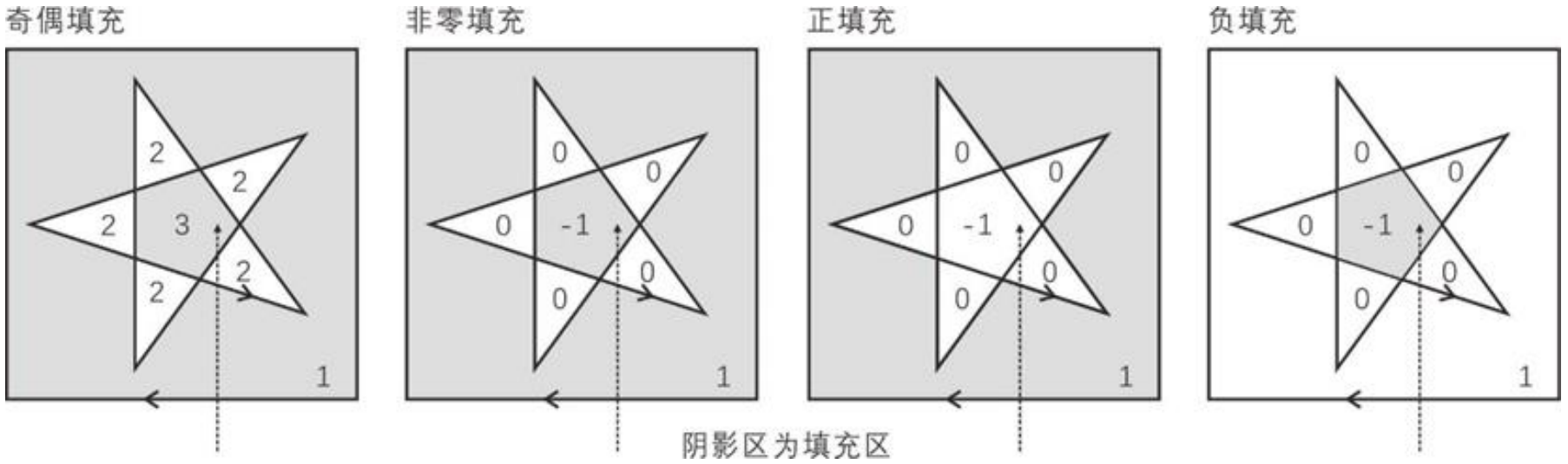


填充时每个区域绕数统计示例

3. 基于裁剪的轮廓偏置

- **3.1. Clipper简介：基本概念——填充规则**
  - **非零填充**：绕数不为0的区域为填充区域。
  - **正填充**：绕数为正的区域为填充区域。
  - **负填充**：绕数为负的区域为填充区域。
  - **奇偶填充**：……

上面三种填充规则适用于那些轮廓存在相交情况的平面区域



填充时每个区域绕数统计示例

3. 基于裁剪的轮廓偏置

- **3.1. Clipper简介：基本概念——subject & clip**

在Clipper库中，**输入轮廓**按其作用可被分为两个类别，一类是**subject**，一类是**clip**。结合4种布尔操作对subject和clip轮廓的作用分别讨论。

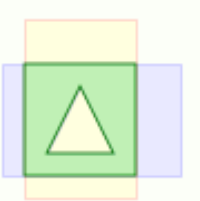
① **求交（intersection或and）**：clip轮廓相当于裁剪窗口，位于窗口外的subject轮廓都将被裁剪并舍弃，而位于窗口内的subject轮廓将被裁剪并保留，形成输出轮廓。

- 对两组**封闭轮廓**来说，求交的结果就是两个组轮廓同时覆盖到的平面区域，因此此时subject和clip是**等价可交换**的；
- 对一组闭合轮廓和一组非闭合轮廓来说，只能由封闭轮廓作为clip（窗口），非封闭轮廓作为subject。

② **求并（union或or）**：subject和clip轮廓也是等价的。求并操作的结果是两组轮廓覆盖区域的和区域。

③ **求差（difference）**：如果用代数减法来类比，**subject**轮廓充当**被减数**的作用，**clip**充当**减数**作用，即**subject – clip**，因此，最终的输出区域相当于从subject区域中抠除掉clip区域的区域。

④ **异或（xor）**：subject和clip轮廓也是等价的。异或操作的结果是输出两组区域并区域中抠除交区域的剩余区域。



3. 基于裁剪的轮廓偏置

• 3.1. Clipper简介：基本概念——函数使用

- Clipper库中最为核心的类是**Pyclipper**类，它封装了关键接口函数AddPath(s)、Execute(2)，其中AddPaths是AddPath函数的复数形式，用于一次性添加多条轮廓，Execute2是Execute的重载函数，它们的功能相同但输出轮廓的组织形式有所差异。示例代码如下：

Test_clipper.py　Python版的Clipper库布尔运算测试　8-3

```
1  from pyclipper import *                          # 导入pyclipper模块
2  from VtkAdaptor import *                         # 导入可视化模块
3  def tuplesToPoly(tuples):                        # 定义将元组序列转化为Polyline函数
4      poly = Polyline()
5      for pt in tuples:
6          poly.addPoint(Point3D(pt[0], pt[1], 0))      # 多边形z坐标始终为0
7      poly.addPoint(poly.startPoint())
8      return poly
9  if __name__ == '__main__':                       # 测试main函数入口
10     subject = [(0,0), (100,0), (100,70), (0,70)]     # 创建subject轮廓subject，正方形
11     clip = [(30,50), (70,50), (70,100), (30,100)]    # 创建clip轮廓，长方形
12     clipper = Pyclipper()                            # 定义Pyclipper对象clipper
13     clipper.AddPath(subject, PT_SUBJECT, True)           # 向clipper添加subject轮廓
14     clipper.AddPath(clip, PT_CLIP, True)             # 向clipper添加clip轮廓
15     sln = clipper.Execute(CT_UNION)                  # 执行union操作，返回结果sln
16     vtkAdaptor = VtkAdaptor()                        # 定义可视化对象
17     for tuples in sln:                               # 转换轮廓数据格式并显示
18         poly = tuplesToPoly(tuples)
19         vtkAdaptor.drawPolyline(poly).GetProperty().SetColor(0, 0, 0)
20     vtkAdaptor.display()
```

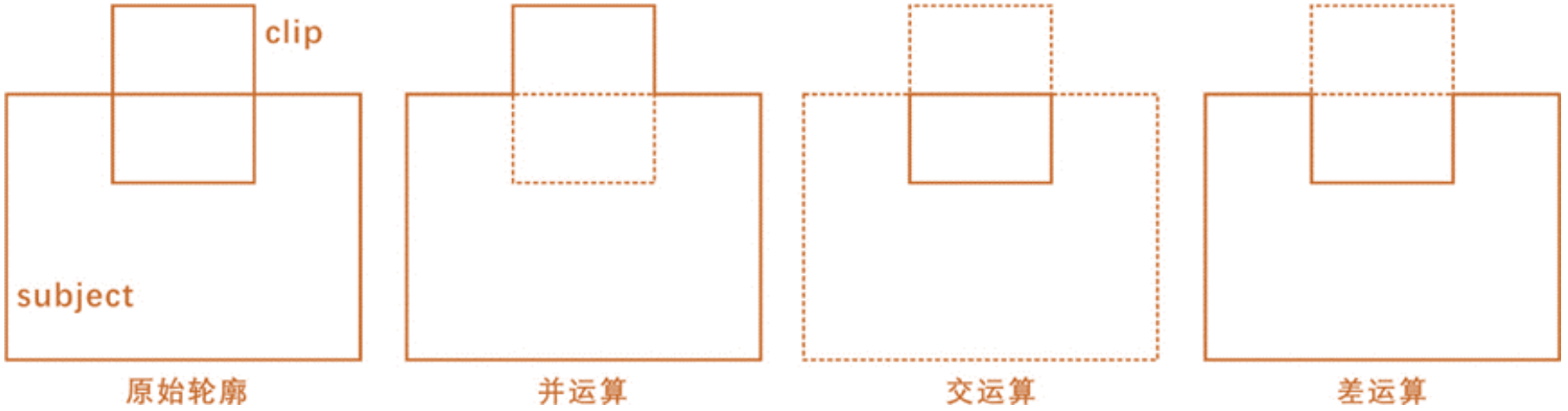
核心操作

3. 基于裁剪的轮廓偏置

Clipper各种裁剪算法的时间复杂度接近线性，速度非常快，且非常稳定

• 3.1. Clipper简介：基本概念——函数使用

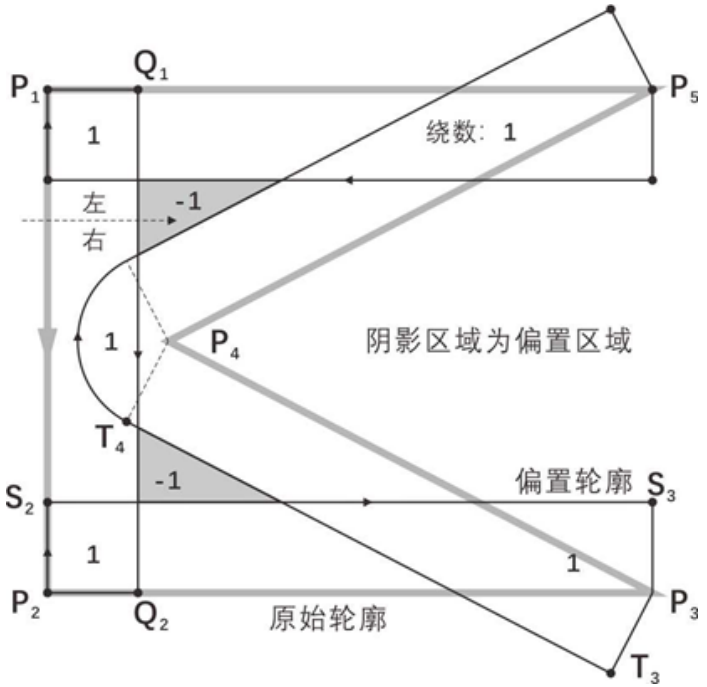
- **步骤**：1）创建clipper对象；2）通过AddPath添加轮廓，并告知轮廓类型是PT_SUBJECT还是PT_CLIP（前缀PT是Polygon Type的缩写）；3）通过调用Execute函数获取裁剪结果。
- 以上代码执行的布尔运算类型为**CT_UNION**（并）。当然也可以尝试在Execute函数中输入**CT_INTERSECTION**（交）、**CT_DIFFERENCE**（差）或**CT_XOR**（异或）执行其他类型的布尔操作。这里前缀CT是Clip Type的缩写，即裁剪类型。
- **Execute函数第2、3个参数的作用是分别指定subject和clip轮廓的填充类型**，包括**PFT_EVENODD**、**PFT_NONZERO**、**PFT_NEGTIVE**、**PFT_POSITIVE**，分别对应四种填充规则，其中前缀PFT是Polygon Fill Type的缩写，即填充类型。如果不指定，**Execute函数默认采用奇偶填充规则**。
- **注意**：Clipper对闭合Path的定义和Polyline对闭合的定义是有所差别！



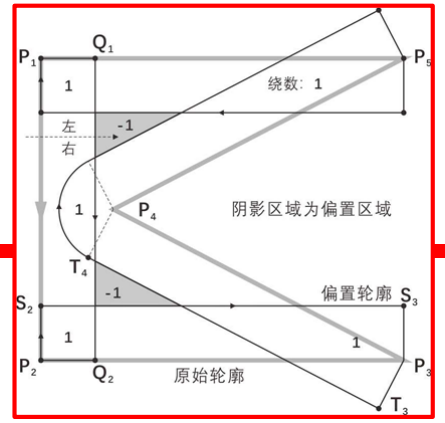
几种布尔运算输出轮廓

3. 基于裁剪的轮廓偏置

- **3.2. 布尔偏置原理：初始偏置曲线生成**
- **线段平移**：以逆时针轮廓的向内偏置为例，如图，对原始封闭轮廓上的连续顶点P_1~P_5，按照线段平移方法首先生成各条原始线段的偏置线段，如P_1P_2的偏置线段为Q_1Q_2，P_2P_3的偏置线段为S_2S_3，P_3P_4的偏置线段为T_3T_4等。
- **劣角偏置连接策略**：当原始轮廓上3个连续顶点所夹内角为劣角（小于180度）时，如顶点P_2，直接连接Q_2P_2和P_2S_2，生成$Q_1Q_2P_2S_2S_3$这一段带有自缠绕效果的连续偏置轮廓，这样将在P_2附近得到一个自相交环。同理，对顶点P_3、P_5、P_1，也可以得到自缠绕连续偏置轮廓，同时也将在顶点附近得到自相交环。



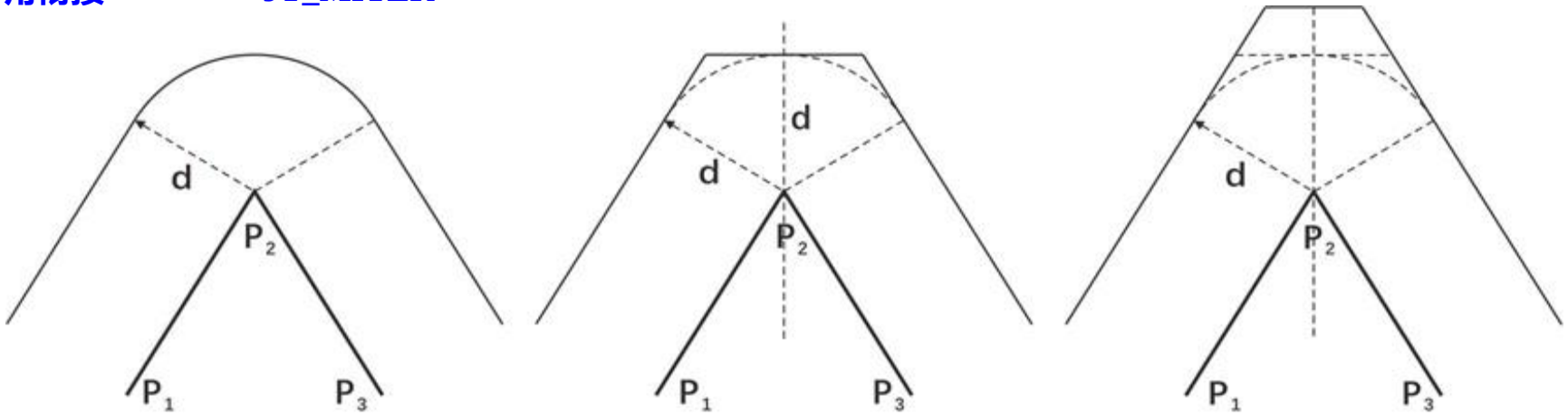
3. 基于裁剪的轮廓偏置



- **3.2. 布尔偏置原理：初始偏置曲线生成**

优角偏置修补策略

- 当原始轮廓上3个连续顶点所夹的内角为优角（大于180度）时，如右图顶点P_4，Clipper中布尔偏置采用了三种策略来衔接相邻偏置线段间的缝隙：
  - ✓ **圆弧衔接** **JT_ROUND**
  - ✓ **矩形衔接** **JT_SQUARE**
  - ✓ **斜角衔接** **JT_MITER**



圆弧离散成弦

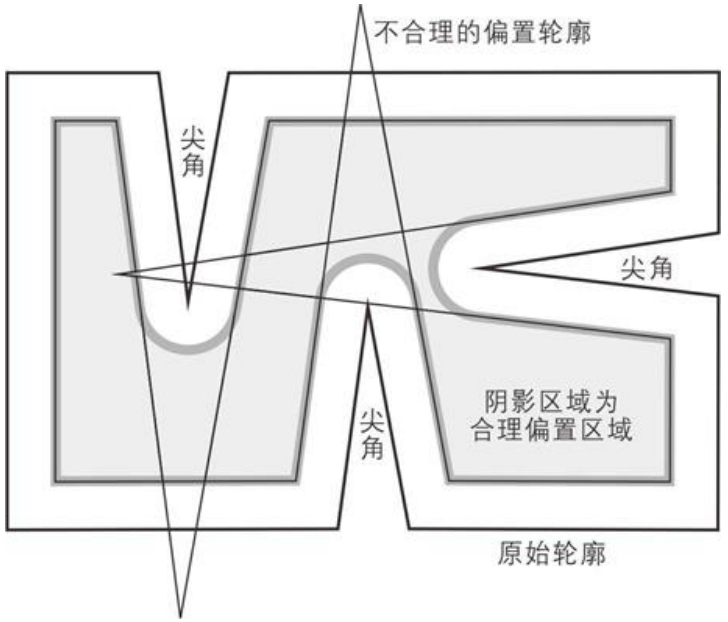
轮廓顶点到衔接线段的距离不大于某一给定值
（Clipper中该值默认为偏置距离的2倍）

3. 基于裁剪的轮廓偏置

- **3.2. 布尔偏置原理：初始偏置曲线生成**

优角偏置修补策略

- 在Clipper库中，**默认**的优角顶点偏置线段的衔接方式是**矩形衔接（JT_SQUARE）**。
- Clipper不支持直接延长两相邻偏置线段至相交点的衔接方式。这是因为对轮廓上的尖角来说，这种衔接方式可能引发严重的偏置问题，导致不合理的偏置轮廓自相交，进而导致本应一体的轮廓区域被分割成若干部分。



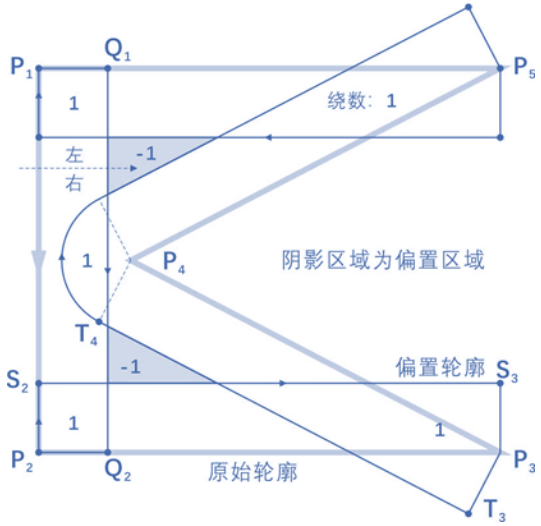
直接连接劣尖角偏置线段引发的不合理分区

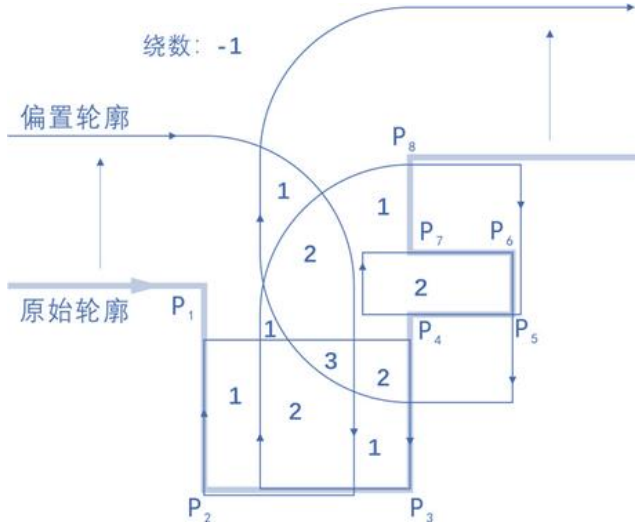
3. 基于裁剪的轮廓偏置

- **3.2. 布尔偏置原理：自相交环消除**

- 在初始偏置轮廓上，可能存在大量的自相交环，特别是在优角顶点附近的局部自相交环。
- **统计各区域绕数**：从各个区域标记的绕数可知，无论是局部还是全局自相交区域内的绕数均为1，而**合法偏置区域内的绕数为–1**，因此，可用四种填充规则中的**负填充**规则来识别并提取合法偏置区域。这一过程需要Clipper库**并（UNION）**布尔运算支持，代码如下：

	初始偏置曲线上负填充区域提取	**8-4**
1	clipper = Pyclipper()	# 定义Pyclipper对象clipper
2	clipper.AddPaths(offsetPolys, PT_SUBJECT, True)	# 添加初始偏置轮廓作为subject
3	sln = clipper.Execute(CT_UNION, **PFT_NEGTIVE**)	# 执行union操作，填充规则为负填充





偏置轮廓自相交区域内绕数统计

3. 基于裁剪的轮廓偏置

- **3.2. 布尔偏置原理：PyclipperOffset**

- Clipper库封装了一个现成可用的轮廓偏置类 **PyclipperOffset**，它的使用方法很简单，示例代码如下：

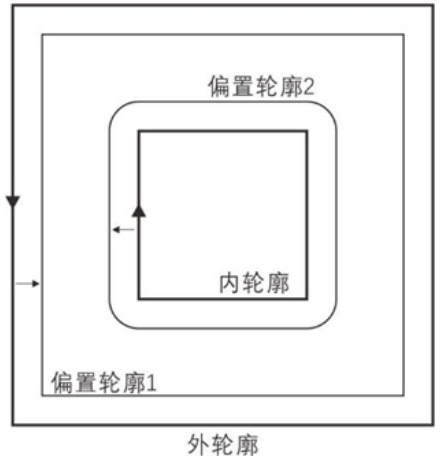
Test_clipperOffset.py	Python版的Clipper库轮廓偏置测试	8-5

```
1  from pyclipper import *                                          # 定义Pyclipper对象clipper
2  outerPoly = [(0,0), (100,0), (100,100), (0,100)]                 # 外轮廓，逆时针
3  innerPoly = [(30,30), (30, 70), (70,70), (70,30)]                # 内轮廓，顺时针
4  pco = PyclipperOffset()                                          # 创建PyClipperOffset对象pco
5  pco.AddPath(outerPoly, JT_ROUND, ET_CLOSEDPOLYGON)               # 添加外轮廓
6  pco.AddPath(innerPoly, JT_ROUND, ET_CLOSEDPOLYGON)               # 添加内轮廓
7  sln = pco.Execute(-7)                                            # 执行内偏置，距离为-7，返回结果sln
```

✓ 首先创建“回”字形轮廓，包括外轮廓outerPoly和内轮廓innerPoly，这两个轮廓都是二维元组的列表。

✓ 然后将这两个轮廓通过PyClipperOffset类的AddPath函数分别加入pco，并在参数中设置偏置线段衔接类型为JT_ROUND，即圆弧衔接，轮廓端点类型为ET_CLOSEDPOLYGON，即封闭多边形。

✓ 最后调用Execute函数生成偏置结果sln，在Execute函数中，设置的偏置距离为–7，**负号表示对逆时针轮廓向内偏置，对顺时针轮廓向外偏置**；如果偏置距离为正，则对逆时针轮廓执行外偏置，顺时针轮廓执行内偏置。



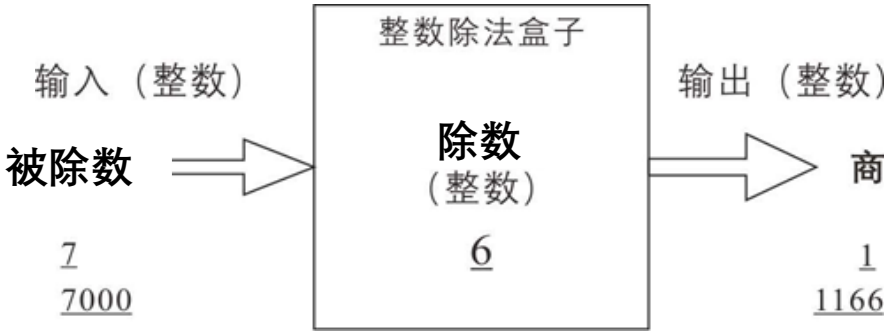
4. Clipper适配器构建

- **数值运算精度问题**

- Clipper中，Path顶点的坐标数据类型为**整型**，所有运算都是整数运算，如加、减、乘、除、平方、开方等。这样设计的优点是可以提高Clipper库在运算时的数值稳定性，缺点是它无法处理比较小的数（如0.1，0.01等）。**Clipper是如何克服这个问题的?**

思想实验：一个只能处理“整数除法” 的小盒子

- ✓ 输入为被除数，输出为商，且除数、被除数、商均为整数。设盒子中的除数为6。
- ✓ 输入被除数7，输出的商为1，和真实结果1.166…还有0.166…的绝对误差。
- ✓ 对输入扩大1000倍，输出为1166，然后对1166除以1000，得到1.166的结果，这个结果已非常接近7除以6的真实值的。
- ✓ 如果对结果的精度还有更高的要求，比如希望保留小数点后7位，那么只需对小盒子的输入乘以10^7，最后对小盒子的输出除以10^7，就能得到具有7位小数的结果。



启发：

对Clipper库的输入和输出，先对输入数值**放大10^N倍**（N为大于0的整数），然后对输出数值**缩小10^N倍**，这样就能够让Clipper处理那些看起来比较小的数，且最终得到的结果能够**保留N位小数**。

4. Clipper适配器构建

- **ClipperAdaptor适配器**
- 直接使用PyClipper类比较繁琐，存在多段线数据类型（Path/Polyline）相互转化的问题，为此，设计并实现了Clipper库的适配器类，如下：

ClipperAdaptor.py	封装Clipper库的数据转化方法及相关函数	8-6

```
1   class ClipperAdaptor:                                   # 定义Clipper适配器类
2       def __init__(self, digits = 7):                     # 初始化函数
3           self.f = math.pow(10, digits)                   # 数值精度，默认为7位小数
4           self.arcTolerance = 0.005                       # 圆弧精度，默认为0.005mm
5       def toPath(self, poly):                             # 将Polyline转化为Path
6           path = []
7           for pt in poly.points:
8               path.append((pt.x * self.f, pt.y * self.f)) # 放大点坐标
9           return path
10      def toPaths(self, polys):                           # 函数toPath的复数形式
11          paths = []
12          for poly in polys:
13              paths.append(self.toPath(poly))
14          return paths
15      def toPoly(self, path, z = 0, closed = True):       # 将Path转化为Polyline
16          poly = Polyline()
17          for tp in path:
18              poly.addPoint(Point3D(tp[0] / self.f, tp[1] / self.f, z))       # 缩小点坐标
19          if len(path) > 0 and closed:                    # 如果封闭，则将起点添加到轮廓
20              poly.addPoint(poly.startPoint())            # 最后一点
21          return poly
22      def toPolys(self, paths, z = 0, closed = True):     # 函数toPoly的复数形式
23          polys = []
24          for path in paths:
25              polys.append(self.toPoly(path, z, closed))
26          return polys
27      def offset(self, polys, delta, jt = JT_SQUARE):     # 偏函数，输入Polyline列表
28          pco = PyclipperOffset()
29          pco.ArcTolerance = self.arcTolerance * self.f   # 指定pco的圆弧精度，放大
30          pco.AddPaths(self.toPaths(polys), jt, ET_CLOSEDPOLYGON)
31          sln = pco.Execute(delta * self.f)               # 偏置距离也须同时放大
32          return self.toPolys(sln, polys[0].point(0).z)   # 返回Polyline列表
```

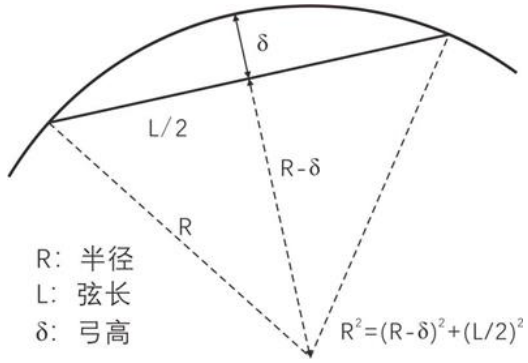
4. Clipper适配器构建

- **ClipperAdaptor适配器**

- 属性**digits**用于指定数值运算精度，即输出结果保留的小数位数，默认情况下，小数点后保留7位有效数字。

- 属性**arcTolerance**用于指定劣角顶点偏置圆弧衔接中的圆弧离散精度。

  - ✓ arcTolerance为离散得到的弦的弓高误差，弓高误差是指弦到真实圆弧的最大距离值，如图，弓高误差越小，弦就越接近真实圆弧，即用弦代替圆弧的精度就越高，但最终得到的弦的总数就越大。在图中，可以得出弓高误差和弦长的关系。
  - ✓ 在Clipper库的官方文档中，有一个表示圆弧精度变量，且它的默认值为0.25个单位，这是相对于Clipper中的整型数据来说的。这里设置arcTolerance的默认值为0.005mm，在具体使用时，需要考虑对该值进行放大处理。

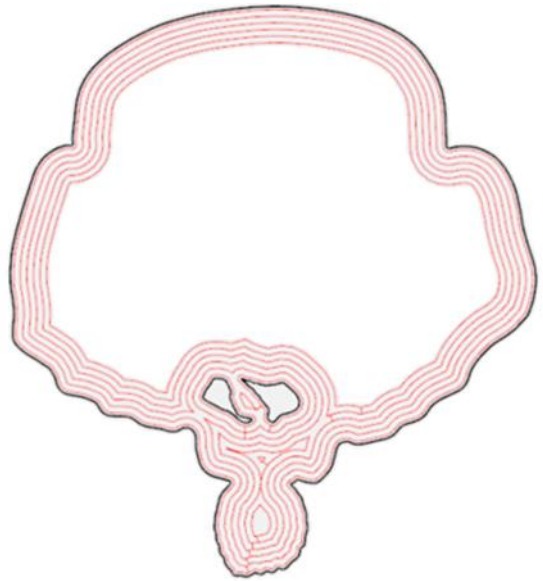


- 函数**toPath(s)**和**toPoly(s)**，实现Polyline和Path的相互转化，其中带‘s’的函数表示转化的多段线为复数形式。
  - ✓ Path为二维平面多段线，而Polyline为三维多段线，因此在将Polyline对象转化为Path对象时，Polyline对象的**z坐标**信息将全部丢失，而将Path对象转化为Polyline对象时，需要从外部额外输入z坐标信息。
  - ✓ Polyline和Path对多段线是否**封闭的定义**不太一致：对Polyline对象，如果它是封闭的，那么它的首尾两个顶点重合；而对Path对象，即便它是封闭的，它的首尾两个顶点也不会重合，只能从相关代码的上下文语境理解Path对象是否封闭。

✓ 函数**offset**的功能是对封闭轮廓执行偏置操作。
  - ✓ 输入为Polyline对象的列表polys、偏置距离delta以及劣角顶点衔接方式，其中对参数delta， 当需对输入轮廓执行向内偏置时，delta为负，反之delta为正；
  - ✓ 输出也是Polyline对象的列表，表示生成的偏置轮廓。

5. 轮廓路径生成与连接

- 面向三维打印的轮廓平行填充路径就是对表示打印区域的边界轮廓连续**偏置**若干次后**连接**得到的路径。
- **两个任务**
  - **路径生成**：根据三维打印的工艺特点生成连续偏置曲线；
  - **路径连接**：连接偏置曲线，得到一条（或若干条）可以遍历这些偏置曲线的打印路径。



5. 轮廓路径生成与连接

• 5.1. 连续偏置曲线生成

- 对打印区域边界轮廓，需要对其连续向内偏置一定次数，以生成填充带。以FDM工艺为例，设填充带厚度为 W，打印机喷头直径为 Ω，则需连续偏置 W/Ω次（取整）。在这一过程中，需要解决以下几个问题。

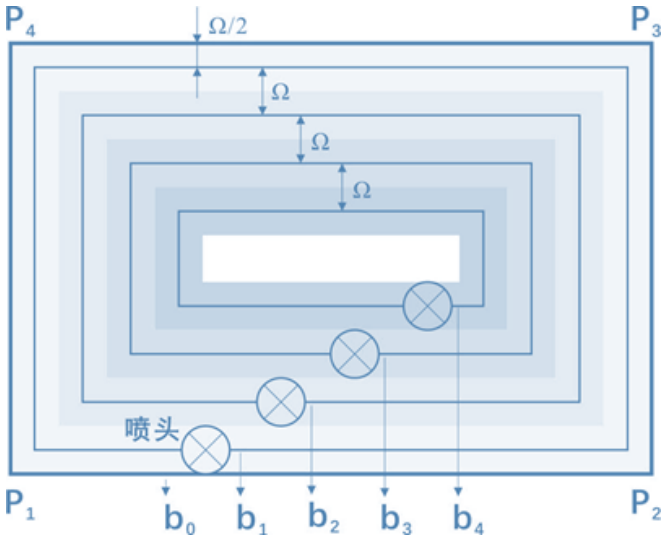
① **偏置距离取值**。丝材宽度为Ω，为保证模型在尺寸上恰好等于设计模型，首次偏置距离应为Ω/2。随后，为了实现相邻偏置路径之间的无缝填充，其他偏置距离应为Ω。理论上，第1次填充后，填充带宽度为Ω；第n次填充后，填充带宽度为nΩ。

② **偏置主体选择**

方案一：基于上一条曲线偏置。为了确保喷头沿着相邻偏置路径填充后丝材边缘恰好靠在一起，不至于重叠或留下缝隙，应基于当前偏置曲线生成下一条偏置曲线，即图中b_1基于b_0偏置Ω/2生成，b_2基于b_1偏置Ω生成，以此类推，b_k基于b_{k-1}偏置Ω生成。

说明

b_i（i为非负整数）曲线可能由一条或多条曲线组成，b_i称为第i级偏置曲线，它是由第i级偏置得到的偏置曲线。

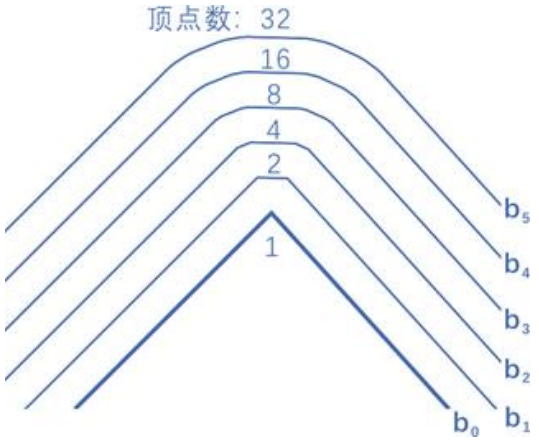


5. 轮廓路径生成与连接

• 5.1. 连续偏置曲线生成

方案一（基于上一条曲线偏置）存在的问题

- **一是偏置顶点数量的无谓递增**
  - 如图，对b_0轮廓上的劣角顶点，在偏置过程中如果采用矩形衔接（Square）的方式，那么在b_1曲线的对应位置上将生成2个劣角顶点；在对b_1进一步偏置得到的b_2曲线上，在相应位置将生成4个劣角顶点……以此类推，在对劣角顶点的连续偏置过程中，在局部区域，偏置曲线顶点数量相对于上一条曲线会成倍增加，而这种顶点数量的增加对偏置轮廓精度的提升是没有意义的，反而会增加后续运算时间。
  - 可以想象，如果采用圆弧衔接（Round）的方式，那么偏置曲线上劣角顶点的数量将会呈爆炸式增长。



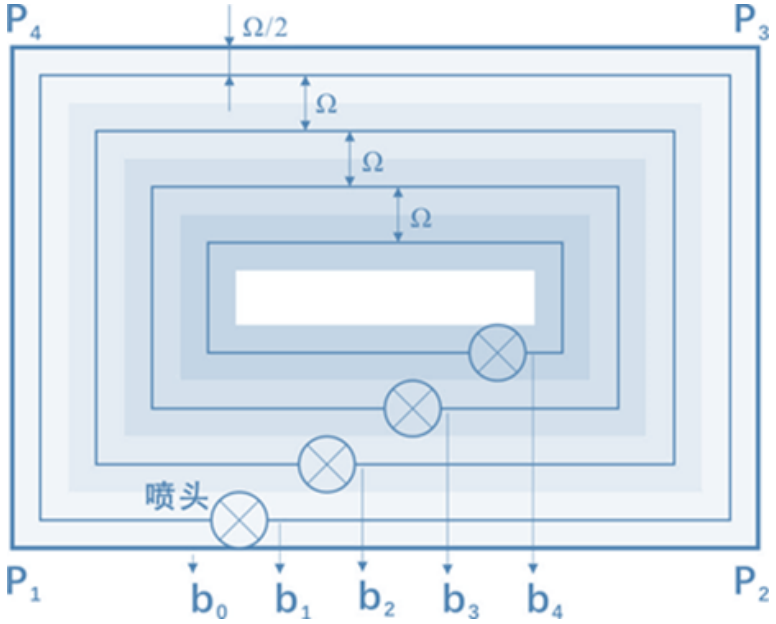
- **二是偏置轮廓精度的下降**
  - 尽管Clipper布尔偏置算法很强大，但它还是无法生成完美的偏置曲线。
  - 在由Cipper生成的偏置曲线上，可能保留了一些轮廓裁剪的的印记，导致偏置曲线在局部区域的微小扰动（如局部微小尖角）。从几何上考虑，这种扰动本身在当前偏置曲线上不会造成什么影响，但是在对当前偏置曲线的连续偏置过程中，这种扰动就可能会被逐渐放大，直至影响偏置轮廓精度。
  - 当然可以在扰动首次产生时对轮廓进行修复，但扰动的识别和消除并不那么容易。

5. 轮廓路径生成与连接

• 5.1. 连续偏置曲线生成

方案二：始终基于边界轮廓偏置

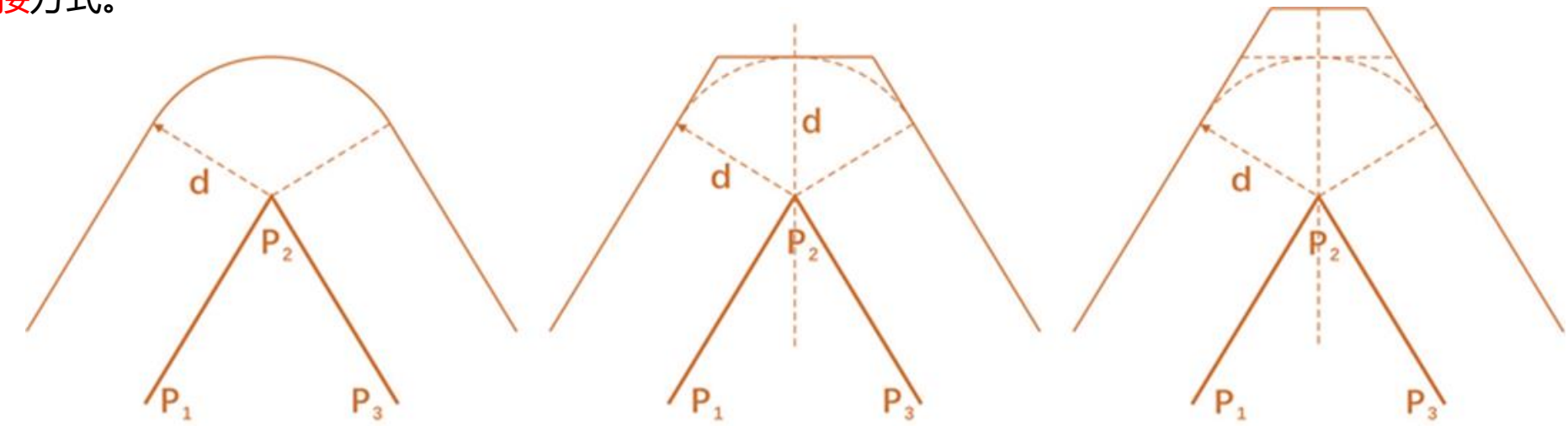
- 每次偏置都是以b_0为原始曲线，如：b_1曲线是对b_0曲线偏置$\Omega/2$距离得到的偏置曲线，b_2曲线是对b_0曲线偏置$3\Omega/2$距离得到的偏置曲线……以此类推，b_k曲线是对b_0曲线偏置$(2k-1)\ \Omega/2$距离得到的偏置曲线。
- 采用这种策略其实是一种妥协，它的缺陷是所有偏置曲线只跟随边界轮廓b_0的形状，相邻偏置线之间形状关联性较小，这样，当喷头沿着相邻偏置线打印后，可能在部分区域留下相互重叠或存在缝隙的丝材。



偏置主体按方案二选择

5. 轮廓路径生成与连接

- **5.1. 连续偏置曲线生成：劣角顶点偏置衔接**
- Clipper库提供了3种劣内角衔接方式，分别是：**圆弧衔接（Round）**、**矩形衔接（Square）**和**斜角衔接（Miter）**，其中圆弧衔接得到的偏置轮廓精度最高，矩形衔接精度次之，斜角衔接精度最差。
- 但圆弧衔接得到的偏置曲线顶点数最多，矩形衔接顶点数次之，斜角衔接顶点数最少，这3种衔接方式各有优缺点，较难取舍，特别是圆弧衔接和矩形衔接两种方式。
- 为了保证路径生成的灵活性，在生成偏置路径时，**将这3种方式都作为可选衔接方式**。但默认情况下，推荐使用矩形衔接方式。



(a) 圆弧衔接 Round　(b) 矩形衔接 Square　(c) 斜角衔接 Miter

5. 轮廓路径生成与连接

• 5.1. 连续偏置曲线生成：代码实现

- 将生成连续偏置曲线的功能封装在GenCpPath类中。

GenCpPath.py	偏置路径生成与连接：连续偏置路径生成（offset）	8-7

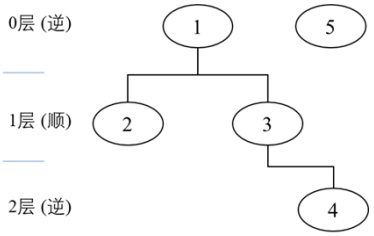
```
1   ……                                          # 导入相关库
2   class GenCpPath:                            # 定义GenCpPath类
3       def __init__(self, boundaries, interval, shellThk):        # 输入：轮廓、偏置距离、填充带宽
4           self.boundaries = boundaries        # 打印区域边界轮廓
5           self.interval = interval            # 工艺参数：偏置距离（喷头直径）
6           self.shellThk = shellThk            # 工艺参数：填充带宽（外壳厚度）
7           self.arcTolerance = 0.005           # 工艺参数：圆弧精度
8           self.jointType = JT_SQUARE          # 工艺参数：衔接类型
9           self.offsetPolyses = []             # 临时存储的中间偏置曲线列表
10          self.paths = []                     # 最终输出的轮廓平行路径列表
11          self.offset()                       # 调用连续偏置路径生成函数
12          self.linkLocalOffsets()             # 调用路径连接函数
13      def offset(self):                       # 定义连续偏置路径生成函数
14          ca = ClipperAdaptor()               # 定义Clipper适配器对象
15          ca.arcTolerance = self.arcTolerance # 设置圆弧精度（使用圆弧衔接）
16          delta = self.interval / 2           # 首次偏置距离
17          polys = ca.offset(self.boundaries, -delta, self.jointType)
18          self.offsetPolyses.append(polys)    # 偏置曲线存放在offsetPolys中
19          while math.fabs(delta) < self.shellThk:     # 循环直至偏置距离大于填充带宽
20              delta += self.interval
21              polys = ca.offset(self.boundaries, -delta, self.jointType)
22              if polys is None or len(polys) == 0: break      # 已到偏置区域中心，则退出
23              self.offsetPolyses.append(polys)
24      def linkLocalOffsets(self): pass        # 定义路径连接函数
```

5. 轮廓路径生成与连接

• 5.1. 连续偏置曲线生成：代码实现

- **输入**：表示打印区域的边界轮廓**boundaries**、偏置距离**interval**和填充带宽**shellThk**三个参数，其中偏置距离就是喷头直径，填充带宽类似于打印后模型的表皮或外壳厚度，这两个参数是和三维打印工艺直接相关的参数，即工艺参数。
- 圆弧精度**arcTolerance**和劣角顶点衔接类型**jointType**两个工艺参数，默认情况下，圆弧精度为0.005mm，劣角顶点采用矩形方式衔接。
  - 注意：在ClipperAdaptor类中已经定义过两个同名参数（属性），尽管它们的默认值一致，但在那个类中定义这两个参数单纯是出于几何上的考虑，而在GenCpPath类中定义这两个参数是出于工艺上的考虑，两者的层次不同。
- 函数**offset**和**linkLocalOffsets**分别用于连续偏置路径生成和连接，但这里只实现了offset函数，linkLocalOffsets函数将在后面实现。
  - offset函数一个循环中反复调用ClipperAdaptor类的offset方法生成输入轮廓的连续偏置曲线。这个循环有两个退出条件：一是当前偏置距离大于指定的填充带宽，偏置过程自然停止；二是最后生成的偏置曲线已到偏置收敛的区域中心，返回空的偏置结果。第二种情况发生的原因是填充带宽设置过大或打印区域过小，生成的偏置曲线已填充整个打印区域。
  - offset函数连续偏置**输出**的偏置曲线存放在临时变量**offsetPolyses**中，注意，这是一个存储Polyline对象的二维列表，其中第一个维度表示偏置级数，第二个维度表示当前偏置级上的曲线序号。
  - 对offsetPolyses，最终目标是将其中的每条偏置曲线串连成一体，以生成轮廓平行填充路径，而且为了保证实际打印效率，用于连接连续偏置曲线的路径长度越短越好。为此，必须首先找出offsetPolyses中各条曲线的**包含关系**。

5. 轮廓路径生成与连接



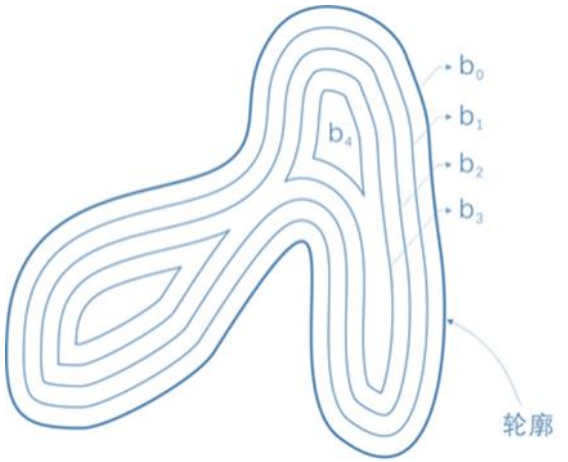
- **5.2. 偏置曲线包含关系确定：建立曲线父子关系**
- **情形一、当 b_0 仅有外轮廓曲线**
  - offsetPolyses的第一个维度表示偏置级数，根据连续偏置的原理，第1级曲线必然包含第2级曲线，第2级曲线必然包含第3级曲线……第i级曲线必然包含第i+1级曲线，即offsetPolyses[i+1]中各条曲线的直系父曲线必然在offsetPolyses[i]中，而offsetPolyses[i]中各条曲线的直系子曲线也必然在offsetPolyses[i+1]中。

    注意：一条曲线可能被多条曲线所包含（即在多条曲线内部），其**直系父曲线是指包含它的曲线中距它最近的那条曲线**，而它本身可称为父曲线的子曲线。根据这一定义，**一条曲线至多有1条直系父曲线**，但可能有0条、1条或多条直系子曲线。通过构建父子关系网，可以通过一条曲线索引到关系网中的所有曲线。

  - 对offsetPolyses[i+1]中的任意一条曲线c，它的直系父曲线必然在offsetPolyses[i]中，如果它在offsetPolyses[i]中某条曲线p的内部，那么p就是c的直系父曲线，c就是p的直系子曲线。

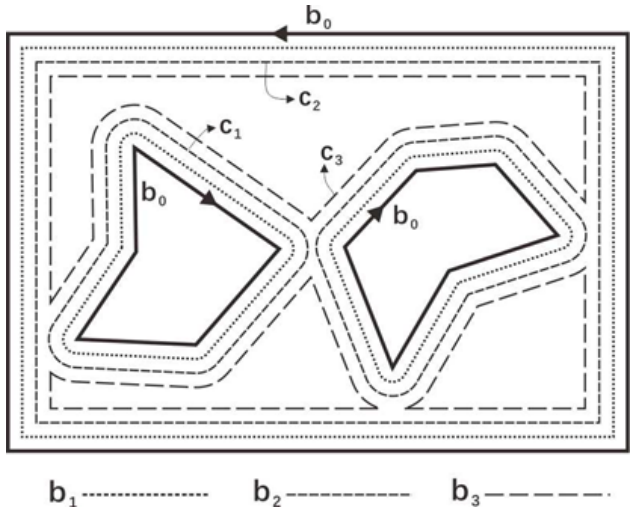
如何如何判断一条曲线在另一条曲线的内部?

考虑到offsetPolyses中所有曲线不相交，因此，任意曲线A是否在另一曲线B内部的问题等价于曲线A上的任意一点（如起点）是否在曲线B内部，而判断一点是否在曲线内部的问题早已解决（参见GeomAlgo中封装的pointInPolygon函数）。



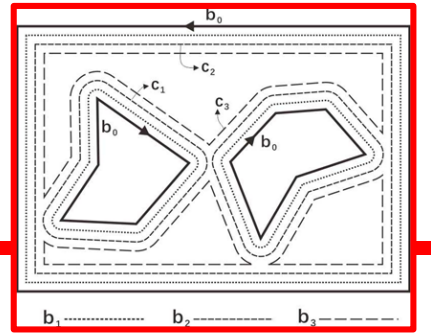
5. 轮廓路径生成与连接

- **5.2. 偏置曲线包含关系确定：建立曲线父子关系**
- **情形二、当 b_0 同时有内外轮廓**
  - **命题：offsetPolyses[i+1]中各曲线的直系父曲线必然在offsetPolyses[i]——不成立！**
    - 如图，曲线c_1和c_2同属第2级偏置曲线b_2，但事实上，c_1位于c_2内部，即c_2是c_1的父曲线；曲线c_3属于第3级偏置曲线b_3，但c_1和c_3同为c_2子曲线……
    - 对offsetPolyses中的各曲线，很难推断它是外轮廓还是内轮廓的偏置曲线。
    - offsetPolyses中曲线之间包含关系的变得繁琐，offsetPolyses的数据结构特点无法利用。



包含内外轮廓的连续偏置

5. 轮廓路径生成与连接



- **5.2. 曲线包含关系确定**

更为一般的命题：

对一维封闭曲线序列polys，已知所有曲线间不相交，确定各曲线之间的包含关系。

- 一般曲线序列polys，偏置生成，或通过其他方式生成。
- 各个曲线不自相交，所有曲线间也不相交。

- **暴力法基本思路**
  - 对polys中的任意一条曲线c，从polys中找出包含它的所有曲线，并把这些曲线放在一个临时列表变量中（记为parents），显然，parents中的所有曲线都是曲线c的潜在父曲线。
  - 根据父曲线的定义，只需从parents找出距离曲线c最近的那条曲线（记为p），曲线p即为曲线c的父曲线，反过来，曲线c即为曲线p的其中一个子曲线。
  - 同理，对polys中的其他曲线，都可以按照以上方法找出它的父曲线，从而构建所有曲线之间的父子关系。
  - 时间复杂度为$O(n^2)$，其中n为polys中的曲线数量，需要进行n^2次的pointInPolygon运算和n^2次的**多边形之间最近距离**计算。

如何寻找距离目标多边形最近的多边形?

5. 轮廓路径生成与连接

更为一般的命题：

对平面封闭曲线序列polys，已知所有曲线间不相交，确定各曲线之间的包含关系。

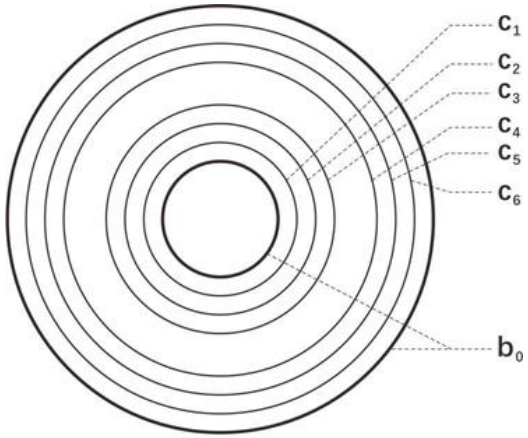
- 一般曲线序列polys，偏置生成，或通过其他方式生成。
- 各个曲线不自相交，所有曲线间也不相交。

• 5.2. 偏置曲线包含关系确定：建立曲线父子关系

一个发现：在封闭曲线的包含关系中，父曲线的面积必然大于子曲线的面积！

• 改进法思路

- 在暴力算法中，对曲线c的潜在父曲线parents，只需计算parents中每条曲线的面积，面积最小的那条曲线就是曲线c的直系父曲线，从而规避多边形距离计算。
  - 如图，曲线c_1的潜在父曲线有c_2、c_3、c_4、c_5、c_6共5条曲线，而在这些潜在父曲线中，c_2面积最小，因此c_2就是c_1的父曲线。
  - 如果一开始便计算polys中各曲线的面积，并按面积对曲线进行排序，还可降低pointInPolygon至少一半的运算次数。因为**对任意曲线polys[i]，只需从面积比它稍大的曲线中（从polys[i+1]开始）寻找它的父曲线。**在运气最好的情况下，只需n次pointInPolygon运算；在运气最差的情况下，需要n(n–1)/2次pointInPolygon运算。



面积比较：$c_1 < c_2 < c_3 < c_4 < c_5 < c_6$

5. 轮廓路径生成与连接

• 5.2. 曲线包含关系确定：代码实现

- 封装一个用于确定多边形父子关系的类，类名为**PolyPcrSeeker**。

PolyPcrSeeker.py	多边形父子关系确定类，确定任意多边形序列父子（包含）关系	8-8

```
1   class PolyPcrSeeker:                                # 定义PolyPcrSeeker类
2       def __init__(self, polys):                      # 初始化函数，输入曲线列表
3           self.polys = Utility.makeListLinear(polys)  # 转化输入列表为线性列表
4           self.seek()                                 # 调用类核心函数
5       def seek(self):                                 # 定义寻找父子关系的核心函数
6           polys = self.polys                          # 仅仅是方便书写
7           for poly in polys:                          # 遍历输入polys
8               poly.area = math.fabs(poly.getArea())   # 动态定义曲线面积属性
9               poly.parent = None                      # 动态定义曲线的父曲线属性
10              poly.childs = []                        # 动态定义曲线的子曲线属性
11              poly.depth = 0                          # 动态定义曲线的深度属性
12          polys.sort(key = lambda t: t.area)          # 根据面积对曲线排序
13          for i in range(0, len(polys) - 1, 1):       # 第1重循环，i从0开始
14              for j in range(i + 1, len(polys), 1):   # 第2重循环，j从i+1开始
15                  pt = polys[i].startPoint()          # 取第1条曲线的起点为测试点
16                  if pointInPolygon(pt, polys[j]):    # 测试点是否在第2条曲线内部
17                      polys[i].parent = polys[j]      # 指定父曲线
18                      polys[j].childs.append(polys[i])# 添加子曲线
19                      break                           # 退出第二重循环
20          for poly in polys:                          # 遍历polys，计算每条曲线深度值
21              self.findPolyDepth(poly)
22          polys.sort(key = lambda t: t.depth)         # 依据曲线深度值对polys排序
23      def findPolyDepth(self, poly):                  # 定义曲线深度值计算函数
24          crtPoly = poly
25          while crtPoly.parent is not None:           # 如果当前曲线的父曲线不为空
26              crtPoly = crtPoly.parent                # 则令当前曲线为父曲线
27              poly.depth += 1                         # 并自增原曲线的深度值
28  def seekPolyPcr(polys):                             # 定义类使用的全局接口函数
29      return PolyPcrSeeker(polys).polys
```

5. 轮廓路径生成与连接

• 5.2. 曲线包含关系确定：代码实现

- PolyPcrSeeker类的**输入**为Polyline类型的曲线列表，注意，曲线列表可以是一维、二维甚至是任意维的，以便兼容offsetPolyses的数据格式，同时保证PolyPcrSeeker类本身的通用性。
- 这是通过**Utility**模块中的**makeListLinear**函数实现的，它能够将任意维度的列表以递归的方式转化为一个一维列表。

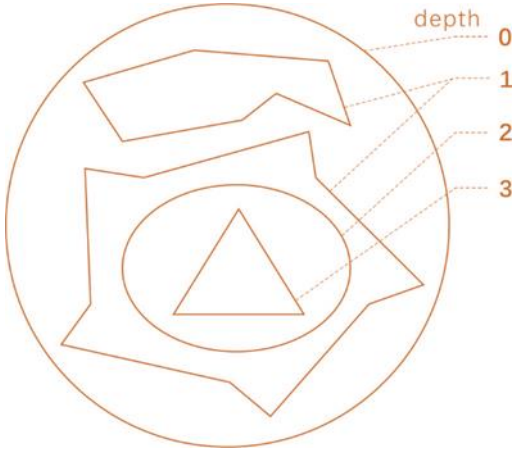
Utility.py	通用功能模块。添加makeListLinear函数，作用是将多维列表转为为一维列表	8-9

```
1   ……                                  # 导入相关库
2   def makeListLinear(lists):          # 定义makeListLinear函数，功能
3       outList = []                    # 是将多维列表转化为一维列表
4       _makeListLinear(lists, outList)
5       return outList
6   def _makeListLinear(inList, outList):   # 定义makeListLinear依赖函数
7       for a in inList:                # 这个函数通过递归的方法将多维
8           if type(a) != list:         # 列表转化为一维列表
9               outList.append(a)       # 注意：我们一般不会在外部调用带
10          else:                       # 下划线的函数
11              _makeListLinear(a, outList)
```

5. 轮廓路径生成与连接

• 5.2. 曲线包含关系确定：代码实现

- **动态属性：area、parent、childs和depth　　——seek函数的关键**
  - 属性**area**表示曲线所围面积，考虑到通过Polyline类的getArea函数计算的面积可能为正也可能为负（面积为正时曲线方向为逆时针，反之为顺时针），而这里的面积表示曲线所围区域的大小，须为非负，因此在给area赋值时要取getArea函数返回值的绝对值。
  - 属性**parent**表示当前曲线的父曲线，它的值可为None，当值为None时，表示当前曲线无父曲线，即它是最外层曲线。
  - 属性**childs**表示当前曲线的的子曲线列表，当列表为空时，表示当前曲线无子曲线。
  - 属性**depth**表示当前曲线所在的深度，，如图，如果一条曲线为最外层曲线，即它的parent属性为None，那么定义它的depth为0，它的子曲线depth为1，子曲线的子曲线depth为2，以此类推。显然，depth越大，表示曲线越在内层，depth越小，表示曲线越在外层。
  - 在PolyPcrSeeker类中，使用**findPolyDepth**函数获取当前曲线的深度值，这个函数的原理是从当前曲线出发，自下而上递归统计父曲线的个数，每统计到一条父曲线，则对当前曲线的depth值自增1，直至最外层曲线。
  - 在seek函数中，首先对polys中每条曲线的area属性赋值，然后据此对polys进行排序。对按面积从小到大排好序的polys的第i条曲线，如果它有父曲线，那么它的父曲线必然是从i+1条曲线开始往后的首条包含它的曲线。据此，可以找到所有曲线的父曲线和子曲线，即对所有曲线的parent和childs属性赋值。
  - 在seek函数的最后，根据得到的父子关系，计算了每条曲线的depth值，并对polys按depth值进行了升序排序。这样做可以让seek函数输出的polys更加有序，外层曲线在列表前面，内层曲线在列表后面。



5. 轮廓路径生成与连接

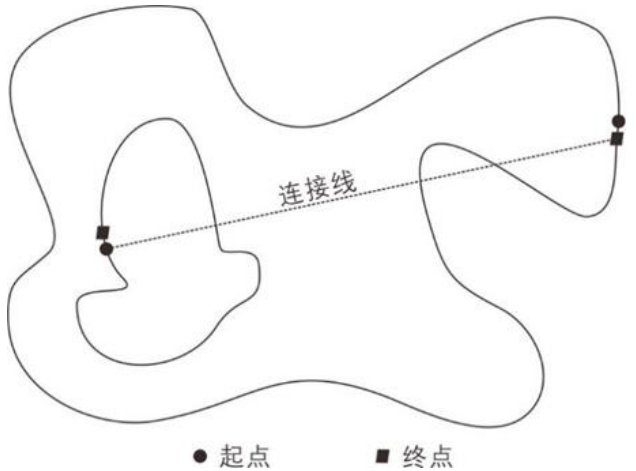
- **5.2. 曲线包含关系确定**
- **全局接口函数seekPolyPcr：**输入为平面封闭曲线列表（一维或多维），输出为已经构建了父子关系且已排好序的曲线列表（一维）。
- 需要说明的是：尽管在PolyPcrSeeker类中，对暴力的曲线父子关系确定方法进行了优化，降低了不少计算量，但从理论上来说，以上seek函数的时间复杂度仍为$O(n^2)$，其中n为输入的曲线数量。
- 回到一开始的问题，对连续偏置得到的曲线列表**offsetPolyses**，只需将其输入seekPolyPcr函数，就能得到相应的已经构建了父子关系的曲线列表。

5. 轮廓路径生成与连接

• 5.3. 偏置路径连接

- **问题提出**：对连续偏置曲线offsetPolyses，当三维打印机喷头沿当前偏置曲线走完一周后，需要跳到另一条偏置曲线上，此时，需要连接这些偏置曲线。
- **直观思路**：是否可以依据offsetPolyses中各曲线的序号直接连接所有曲线，并将得到的曲线作为驱动打印机喷头运动的路径呢？
  - 使用Clipper库偏置得到的偏置曲线的起点（终点）位置是不确定的，它可能在曲线左侧，也可能在曲线右侧。
  - 对两条相邻级别的偏置曲线，连接第一条曲线终点和第二条曲线起点的直线段可能特别长，甚至跨越整个轮廓区域。
  - 对两条在同一偏置级别上的曲线，即便我们可以各自调整它们的起点终点位置，最终得到的连线线段仍然可能非常长。
  - 对实际三维打印过程来说，较长的路径连接线段意味着较长的喷头移动时间，而在这段时间内喷头只空走不出丝，它对整个打印过程是徒劳的，只会延长打印时间，因此在路径规划时我们应该尽量减少这种喷头空走时间，降低路径之间连接、跳转线段的长度。显然，采用直接按照序号连接偏置曲线的策略无法得到最短的打印路径。

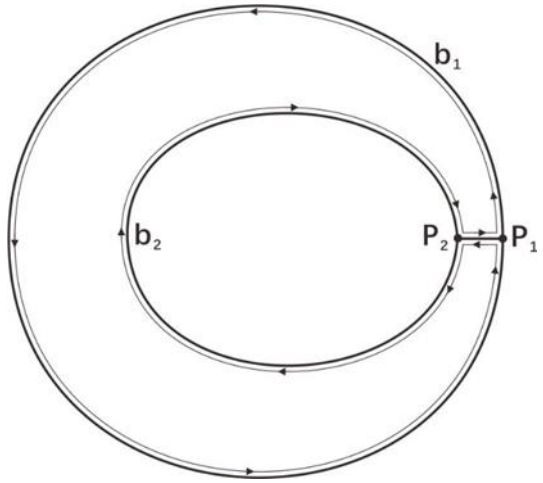
直接连接两曲线端点存在的问题



5. 轮廓路径生成与连接

• 5.3. 偏置路径连接

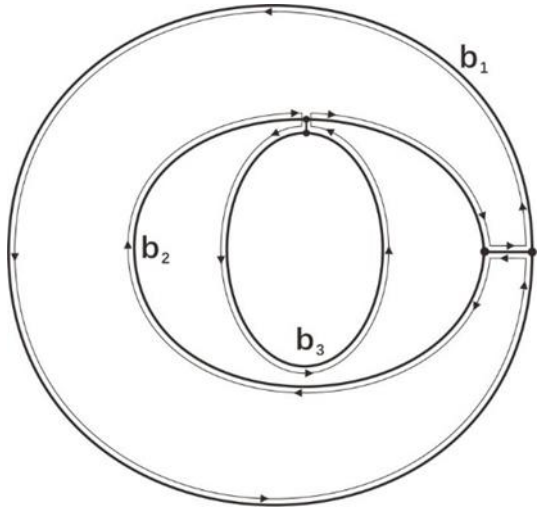
- **问题抽象**：对一系列连续偏置曲线，寻找一种策略，连接所有曲线并得到最短的打印路径。
- **2条相邻包含曲线的连接　　——b_1包含b_2**
  - 这个问题相比于原问题要直观、简单，只需要从两条曲线上找到距离最近的两点连接即可。
  - 具体地，假设P_1和P_2是曲线b_1和b_2上距离最近的两点，那么这两条曲线的连接方式如图所示。连接之后得到的新曲线的起点为曲线b_1起点，它沿着b_1走向到P_1点后连至P_2点，然后从P_2点出发沿着曲线b_2走向绕曲线一周回到P_2点（具体是先到b_2终点，再从b_2起点出发沿着b_2到P_2点），随后再从P_2点连出至P_1点，沿着曲线b_1走向值b_1终点。
  - 根据以上连接方法，可将两条相邻曲线连成一体，得到一条新曲线。



2条具有包含关系的曲线连接方法

5. 轮廓路径生成与连接

- **5.3. 偏置路径连接**
- **3条相邻包含曲线的连接　　——b_1包含b_2，b_2包含b_3**
  - 首先处理处于内层的两条曲线b_2和b_3，对这两条曲线，按照以上介绍的连接方法将b_3桥接到b_2，并用得到的新曲线替换b_2。
  - 然后，对b_1和新b_2，我采取相同的策略将新b_2桥接到b_1，并记得到的新曲线为新b_1。
  - 新b_1就是包含了所有曲线的最短路径。

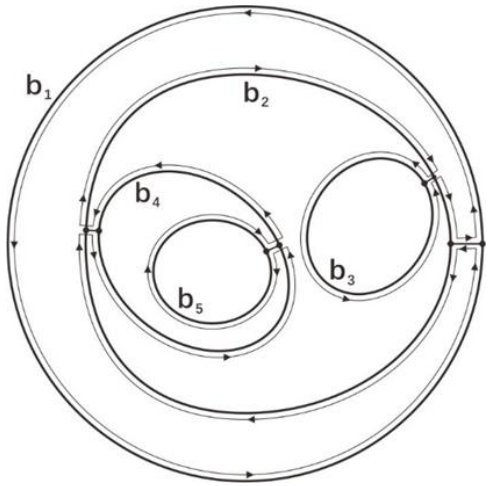


两条具有包含关系的曲线连接方法

5. 轮廓路径生成与连接

- **5.3.偏置路径连接**

- **5条曲线b_1~b_5的连接问题**
  - 包含关系：b_1包含b_2，b_2包含b_3和b_4，即b_3和b_4位于同一级，b_4包含b_5。
  - 首先，从5条曲线中取出处于最内层的曲线b_5及其父曲线b_4，对这两条曲线，按照以上连接策略将b_5桥接至b_4，并用得到的新曲线替换b_4，同时移除b_5。
  - 然后，从剩下的4条曲线中取出处于内最内层的曲线，此时b_3和新b_4同时位于最内层，不妨先取b_3。对b_3和它的父曲线b_2，将b_3桥接到b_2，得到新b_2并移除b_3。
  - 接下来，从剩下的3条曲线（b_1、新b_2、新b_4）中取出处于最内层的曲线新b_4，并将新b_4桥接到新b_2，再次得到一条全新的b_2。
  - 最后，将新b_2桥接到b_1，得到新b_1，新b_1就是包含了所有曲线的最短路径。

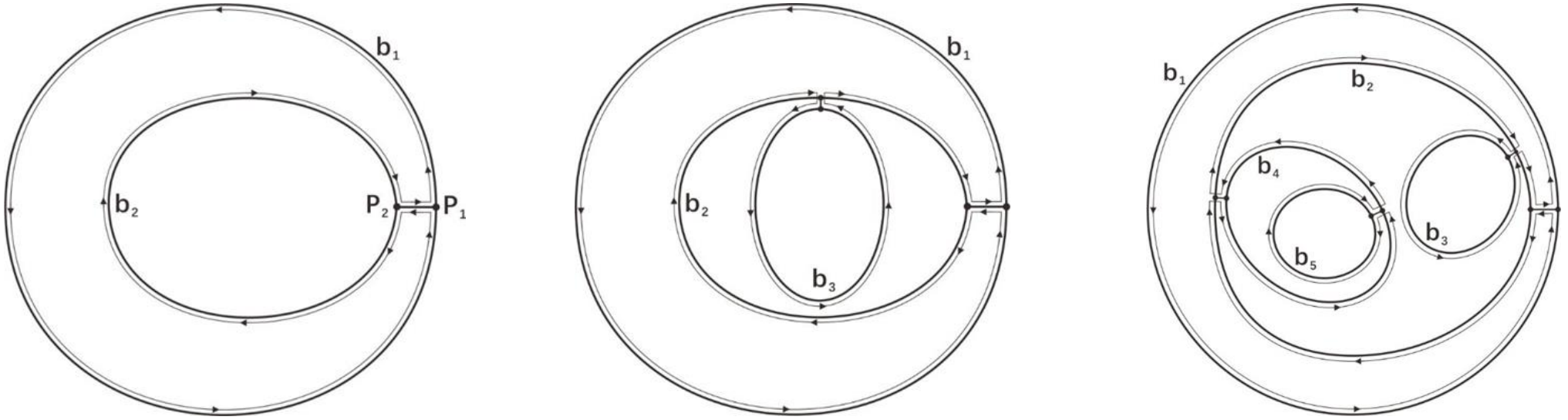


两条具有包含关系的曲线连接方法

5. 轮廓路径生成与连接

- **5.3. 偏置路径连接：连接策略**

- 无论输入曲线的数量有多少，它们之间存在多么复杂的包含关系，在每一步中只要遵循**将处于最内层的曲线在最近点连接到它的父曲线上**的原则，最后总能将所有处于内层的曲线桥接到最外层曲线，从而得到连成一体的最短路径。

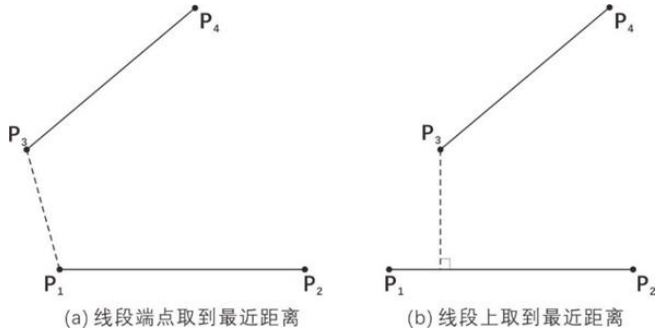


5. 轮廓路径生成与连接

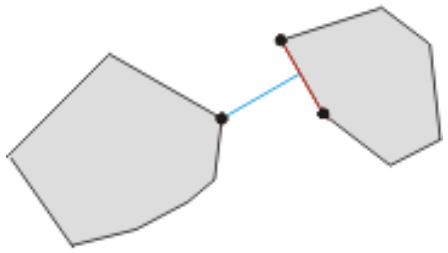
• 5.3. 偏置路径连接：两多段线最近点求取

问题描述：对任意两条不相交的平面多段线，求取它们之间的最近距离以及取到最近距离的两个点。

- 理论上，两条多段线的最近距离不一定就在两个顶点处取到，它也有可能在两条边线段上取到，而后一种情况涵盖了前一种情况，如图所示。
- 如果不计时间成本，只需构造一个双重循环，分别遍历两条曲线中的所有边，两两计算线段之间的距离，从中找出最小距离及最小距离取到时的边序号即可。显然，这种方法的时间复杂度为O(mn)，其中m、n分别为两条多段线的边数。
- 考虑到在偏置曲线连接过程中（特别是还有很多切片层）需要进行密集的多段线最近距离计算，因此，这种方法不太可取。



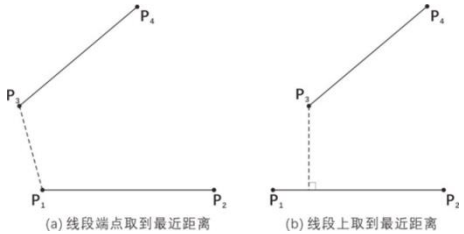
两条线段距离计算



Bridge across islands

Thousands of thousands years ago there was a small kingdom located in the middle of the Pacific Ocean. The territory of the kingdom consists two separated islands. Due to the impact of the ocean current, the shapes of both the islands became convex polygons. The king of the kingdom wanted to establish a bridge to connect the two islands. To minimize the cost, the king asked you, the bishop, to find the minimal distance between the boundaries of the two islands.

5.轮廓路径生成与连接



- **5.3. 偏置路径连接：两多段线最近点求取**

结合实际应用背景中，考虑到快速、精确地计算两曲线的最近点较为繁琐，不妨退而求其次。

- **第一个退步，直接在两多段线的顶点处取最近点**

  理论上，两多段线的最近距离在两边线段上取到，但如果真的那么做，在每一次边对距离单元计算中，需要讨论多种情况，如一条线段的端点投影是否落在另一条线段上，如果是，需要在该投影处将线段一分为二，并以此点作为连接点，这将会使单元计算和连接问题本身变得非常繁琐而费时。

- **第二个退步，直接在其中一条多段线上人为指定一个点为最近点（方便起见，指定起点）**

  这样做可节省一重循环，只要根据这个点在另一条多段线上遍历寻找它的最近点即可。尽管如此，注意，这里的第二重循环是不可少的，否则如果随便指定第二条多段线上的最近点，可能出现连接线段穿越轮廓区域的风险。

通过两步的妥协，可将两多段线最近点的求取过程降到线性时间复杂度！

5. 轮廓路径生成与连接

- **5.3. 偏置路径连接：代码实现**

- 实现GenCpPath类的剩余部分，包括**linkToParent**和**linkLocalOffsets**两个函数，其中前者是后者的依赖函数，linkLocalOffsets的功能是将局部连续偏置曲线连成一体。

GenCpPath.py | 偏置路径生成与连接：连续偏置曲线连接成路径（linkLocalOffsets） | 8-10

```
1   ......                                          # 导入相关库
2   class GenCpPath:                                # 定义GenCpPath类
3     ......                                        # 忽略类中其他代码
4     def linkToParent(self, child):                # 定义连接子曲线至父曲线函数
5       parent = child.parent
6       pt = child.startPoint()                     # 获取子曲线起点pt
7       dMin, iAtdMin = float('inf'), 0             # 使用float('inf')获取浮点数最大值
8       for i in range(parent.count()):             # 寻找父曲线上到pt最近的点
9         d = pt.distanceSquare(parent.point(i))
10        if d < dMin:
11          dMin, iAtdMin = d, i                    # iAtMin为父曲线距pt最近点序号
12      newPoly = Polyline()                        # 新建连接父子曲线的新曲线
13      for i in range(iAtdMin+1):                  # 连接曲线过程
14        newPoly.addPoint(parent.point(i).clone())
15      newPoly.endPoint().w = 1                    # 标记newPoly上连进点w=1
16      for i in range(child.count()):
17        newPoly.addPoint(child.point(i).clone())
18      newPoly.endPoint().w = 1                    # 标记newPoly上连出点w=1
19      for i in range(iAtdMin, parent.count(), 1):
20        newPoly.addPoint(parent.point(i).clone())
21      return newPoly                              # 返回新曲线
```

……

5. 轮廓路径生成与连接

- **5.3. 偏置路径连接：代码实现**

GenCpPath.py	偏置路径生成与连接：连续偏置曲线连接成路径（linkLocalOffsets）	8-10

```
1   ......                                          # 导入相关库
2   class GenCpPath:                                # 定义GenCpPath类
3     ......                                        # 忽略类中其他代码
```

……

```
22    def linkLocalOffsets(self):                   # 定义连续偏置曲线连接函数
23      seekPolyPcr(self.offsetPolyses)             # 首先确定所有曲线的父曲线
24      for i in range(len(self.offsetPolyses) - 1, 0, -1):  # 逐一将所有子曲线连至父曲线
25        childs = self.offsetPolyses[i]
26        for j in range(len(childs)-1, -1, -1):    # 注意range里的参数取值
27          child = childs[j]
28          newPoly = self.linkToParent(child)      # 调用linkToParent函数
29          parent = child.parent                   # 用newPoly中的点替换原parent
30          parent.points = newPoly.points          # 中的点
31          del childs[j]                           # 重要：从列表中删除子曲线
32      for path in self.offsetPolyses[0]:          # 最后将offsetPolyses中的剩余
33        self.paths.append(path)                   # 曲线转存到类的paths属性中
34      self.offsetPolyses.clear()                  # 清空临时列表offsetPolyses
35  def genCpPath(boundaries, interval, shellThk):  # 定义GenCpPath类接口函数
36    return GenCpPath(boundaries, interval, shellThk).paths
```

5. 轮廓路径生成与连接

- **5.3. 偏置路径连接：代码实现**

- **linkToParent**函数的作用是将输入的子曲线（child）桥接到父曲线（parent）上，它的输出是一条包含了父曲线和子曲线上所有有序点的多段线。在这个函数中，简单起见，人为指定了子曲线的起点（pt）为子曲线上所有点到父曲线的最近点，注意，这是一种为了权衡计算量之后的近似处理。

- **linkLocalOffsets**函数是GenCpPath类中除offset函数外的第二个核心功能函数。
  - 在桥接得到新曲线后，用新曲线上的点（Polyline的points属性）替换原父曲线上的点的，而不是直接用新曲线（newPoly）替换父曲线（parent），这样做的原因是为了保持offsetPolyses中曲线元素的地址不变。如果直接用新曲线替换原父曲线，那么将无法再用offsetPolyses索引到新曲线。
  - 在将子曲线桥接到父曲线后，还需注意的是必须从offsetPolyses中移除相应子曲线。
  - 在将各级子曲线桥接到父曲线后，offsetPolyses应该只保留了若干原最外层的曲线地址，只需将这些曲线转存到GenCpPath类的paths属性即可，paths就是GenCpPath最终输出的已经将所有偏置曲线连成一体轮廓路径。

- **genCpPath**函数，GenCpPath类的全局接口函数，以方便后续使用该模块生成轮廓路径。它的输入参数和GenCpPath类的初始化参数完全一致，输出为已连接的路径列表。

5. 轮廓路径生成与连接

• 5.3. 偏置路径连接：全局路径连接

- **paths**中路径数量和原**offsetPolyses**中最外层的偏置曲线的数量一致，这些路径之间的连接只能通过**“跳刀”**的方式实现。
  - 对数控加工来说，跳刀意味着刀具从一条路径的终点抬起至安全平面，然后在安全平面上移动至另一条路径的起点上方，最后下降至另一条路径的起点。跳刀可以避免刀具在大幅移动过程中和工件发生干涉；但跳刀是一种无效切削过程，因此在路径规划中因尽量缩短跳刀路径长度。
  - 对FDM三维打印来说，无需抬起喷头，只需保证在跳刀过程中喷头不出丝材即可。
- **暂不处理paths中各条独立路径的连接问题**
  - paths中各条独立路径的连接问题是一个典型的TSP（Travelling Salesman Problem，旅行商问题）。
  - TSP的定义为：给定一系列城市和每对城市之间的距离，求解访问每一座城市一次并回到起始城市的最短回路（见图8.32）。寻找TSP的最优解非常困难，其中最简单的穷举法的复杂度为O(n!)，其中n为旅行商需要遍历的城市个数。当n仅为50时，需要的枚举次数已超过1064次。所以，大部分优化算法都趋向寻找TSP的近似解。
  - 目前，世界上最快的TSP求解器是丹麦人Keld Helsgaun开发的LKH（http://akira.ruc.dk/~keld/）。除此之外，Concorde（http://www.math.uwaterloo.ca/tsp/）也是一个比较著名的TSP求解器，目前，Concorde提供了Python编程接口。



World TSP（1,904,711 个点）



蒙娜丽莎TSP（10万个点）

6. 轮廓路径生成测试

• 6.1. 切片文件生成

- 对GenCpPath类进行实例测试，测试的对象为实际STL模型的切片文件（即SLC文件）。方便起见，使用SliceAlgo模块的**writeSlcFile**函数生成SLC文件，然后用**readSlcFile**函数读取SLC文件。

Test_saveSlc.py | **生成测试用的STL模型切片文件，即SLC文件** | **8-11**

```
1  from SliceAlgo import *                                  # 导入SliceAlgo模块
2  modelName = "monk"                                       # 指定STL模型文件名
3  layerThk = 2.0                                           # 指定切片厚度：2.0, 1.0, 0.5, 0.2mm
4  src = vtk.vtkSTLReader()                                 # 通过VTK加载STL模型
5  src.SetFileName("E:\\STL\\%s.stl" % modelName)
6  stlModel = StlModel()
7  stlModel.extractFromVtkStlReader(src)
8  layers = slice_topo(stlModel, layerThk)                  # 使用拓扑切片获取每层轮廓数据
9  writeSlcFile(layers, "D:\\%s at %smm.slc" % (modelName, layerThk))          # 写SLC文件
10 print("SLC file has been generated and saved")
```



monk 336,974

- ✓ 测试模型为**monk**该模型包含约33万个三角面片，Z方向高度约为176mm。
- ✓ 对STL模型，使用SliceAlgo模块中的slice_topo函数计算当前模型的切片列表layers。注意：对列表layers中的每一层切片，它包含了一系列表示打印区域边界的平面轮廓，这些轮廓之间的方向是杂乱不定的，需要使用**adjustPolygonDir**函数对每层切片轮廓的方向进行了调整。
- ✓ 设置的切片层厚分别是2.0mm、1.0mm、0.5mm以及0.2mm，得到4个SLC文件。

6. 轮廓路径生成测试

• 6.2. 路径生成测试

- 在得到了切片文件之后，接下来使用GenCpPath类针对这些切片文件生成轮廓平行填充路径。代码如下：

Test_genCpPath.py | **读取SLC文件，生成每层切片的轮廓填充路径** | **8-12**

```
1  from GenCpPath import *                                  # 导入GenCpPath库
2  interval = 0.4                                          # 轮廓路径间距
3  shellThk = 2.0                                          # 外壳厚度，即轮廓路径总宽度
4  pathses = []                                            # 存储各层已连接的路径
5  layers = readSlcFile("D:\\monk at 1.0mm.slc")           # 读取SLC文件
6  start = time.clock()                                    # 计时开始
7  for i in range(len(layers)):                            # 逐层生成轮廓填充路径
8      print('cp, layer: %d / %d' % (i+1, len(layers)))
9      paths = genCpPath(layer[i].contours, interval, shellThk)
10     pathses.append(paths)                               # 存储路径到pathses
11 end = time.clock()                                      # 计时结束
12 print("GenCpPath time: %f CPU seconds" % (end - start))        # 打印路径生成时间
13 va = VtkAdaptor()                                       # 显示，使用VTK库
14 for layer in layers:                                    # 显示每层打印区域边界，黑色
15     for contour in layer.contours:
16         va.drawPolyline(contour).GetProperty().SetColor(0, 0, 0)
17 for paths in pathses:                                   # 显示每层路径，红色
18     for path in paths:
19         va.drawPolyline(path).GetProperty().SetColor(1, 0, 0)
20 va.display()
```

变量interval表示轮廓路径间距，shellThk表示外壳厚度，即轮廓填充路径总宽度，这里设置路径间距为0.4mm，外壳厚度为2.0mm，根据连续偏置原理，GenCpPath类将对每层轮廓偏置5次。

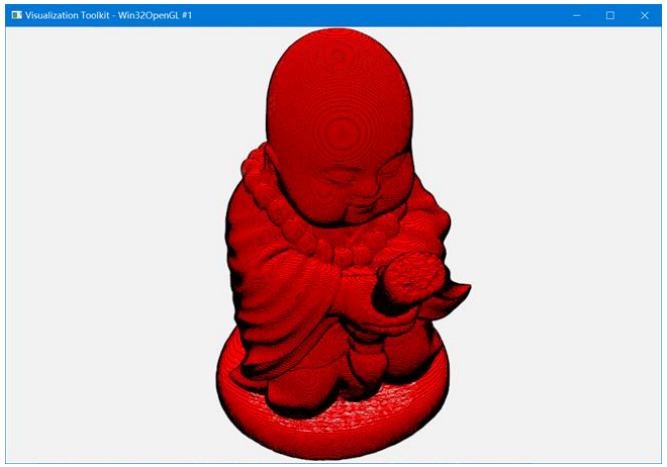
6. 轮廓路径生成测试

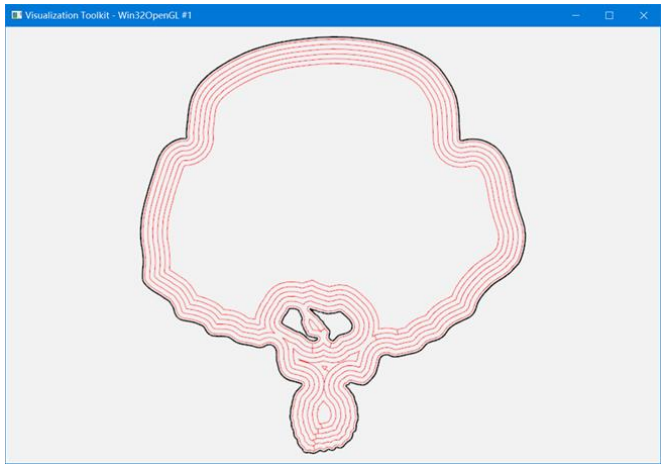
• 6.2. 路径生成测试

- 测试用主机CPU为intel Core i7-8700 @ 3.20GH。在不同切片厚度下的轮廓路径生成时间统计如下：

模型	面片数	模型高度(mm)	切片厚度(mm)	层数	轮廓路径生成时间 (秒)
monk	336,974	175.5	2.0	87	13.1
			1.0	175	26.7
			0.5	351	53.4
			0.2	877	132.9

- ✓ 在切片厚度为0.2mm，接近日常FDM三维打印层厚时，所需轮廓路径生成时间为132.9秒。
- ✓ 相对于常用的三维打印切片软件（如Cura）来说，这样运行速度算是比较慢的。这其实主要是使用的编程语言引起的。C++比Python的效率要高1~2个数量级，即10~100倍。
- ✓ Clipper库也封装了一个检测一个点是否在多边形内部的函数，如果使用这个函数替换GeomAlgo模块中的pointInPolygon函数，轮廓路径生成的效率能提升2倍以上。





本章小结

- **三维打印的常用填充路径包括轮廓路径和平行路径，这一章主要介绍的是轮廓路径的生成原理和方法。**
  - 生成轮廓路径的基础是对表示打印区域的边界进行偏置，首先尝试了基于线段平移的偏置和基于角平分线的偏置，但这两种方法都不能很好地处理局部自相交问题。
  - 为此，引入并详细介绍了一个专门用于平面轮廓裁剪及偏置运算的第三方库Clipper。
  - 为了克服多边形轮廓数据格式的兼容性问题，构建了一个Clipper库的适配器。
  - 在此基础上，首先生成了边界轮廓的连续偏置曲线，然后通过识别各条偏置曲线之间的包含关系，将处于内部的子曲线逐条连接到父曲线上，从而得到若干连成一体的路径曲线，这些路径曲线就是所求的轮廓填充路径。
  - 将生成轮廓路径的流程和功能封装成了一个GenCpPath类，并在最后对这个类进行了测试，结果表明，如果除去编程语言本身的因素，GenCpPath类能在较短的时间内生成轮廓路径。





思考题

1. 在三维打印中，对每层切片，为什么一般使用轮廓平行和方向平行两种填充方式?
2. 分析相邻偏置路径在尖角处的打印材料分布情况。
3. 如果偏置路径上的尖角非常尖锐，会对打印过程造成什么影响？如何光顺路径上的尖角?
4. 对平面曲线的偏置过程，什么是自相交环？如何消除自相交环?
5. 尝试使用Python语言复现文献（Lin Z, Fu J, He Y, Gan W. A robust 2D point-sequence curve offset algorithm with multiple islands for contour-parallel tool path. Computer-Aided Design. 2013; 45(3): 657-70）中的曲线偏置算法。
6. 设计一种算法，寻找平面上任意多边形的最大内接圆（圆心位置及半径大小）。
7. 为什么Clipper库使用整形数表示浮点数？具体如何表示?
8. 简述Clipper库中基于布尔运算的曲线偏置原理。
9. 如果事先未对表示平面区域的轮廓方向进行整理，直接调用Clipper库中的偏置函数对其偏置，会有什么影响?
10. 如何快速检测平面封闭曲线间的包含关系并建立父子继承关系?
11. 说明代码8-10中linkLocalOffsets函数可能存在的问题，如何改进?